



TECNOLÓGICO DE COSTA RICA
ESCUELA DE INGENIERÍA EN COMPUTACIÓN
IC-7841 – PROYECTO DE INGENIERÍA DE SOFTWARE

Documento de Plan de Pruebas

Sistema Web - Biskoto

INTEGRANTES

Daniel Alemán Ruiz | 2023051957

Luis Meza Chavarria | 2023800023

PROFESORA

Ing. Alicia Salazar Hernandez, MSc

PERIODO Y FECHA

Verano 2025-2026

28 de enero 2026

Tabla de contenido

1. INTRODUCCIÓN.....	1
1.1 Propósito	1
1.2 Alcance.....	1
1.3 Entorno y Tecnologías de Pruebas	1
2. ESTRATEGIA Y METODOLOGÍA DE PRUEBAS.....	3
2.1 Niveles de Prueba	3
2.2 Gestión de Datos de Prueba	3
2.3 Entorno de Pruebas.....	4
3. EJECUCIÓN DE PRUEBAS DEL SISTEMA: FRONTEND (CAPA DE PRESENTACIÓN)	5
3.1 Entorno de Ejecución (Vitest).....	5
3.2 Verificación de Lógica de Estado Global (Context API)	6
3.3 Validación de Flujos Críticos de Usuario (Págnas y Formularios)	8
3.4 Pruebas de Componentes de Infraestructura UI	10
4. EJECUCIÓN DE PRUEBAS DEL SISTEMA: BACKEND (LÓGICA Y DATOS)11	
4.1 Entorno de Ejecución y Resumen.....	11
4.2 Pruebas de Integración por Módulos	12
4.2.1 Módulo de Identidad y Seguridad.....	12
4.2.2 Módulo de Gestión de Recursos (Inventario, Suministros y Maestros) .	13
4.2.3 Módulo de Transacciones Comerciales (Pedidos)	15
4.2.4 Módulo de Interacción y Contenido.....	16
4.3 Pruebas Unitarias (Lógica Aislada).....	17
4.3.1 Núcleo de Seguridad y Usuarios.....	17
4.3.2 Gestión de Catálogo e Inventario (Core del Negocio).....	22
4.3.3 Cadena de Suministro y Datos Maestros	27
4.3.4 Transacciones y Reglas Comerciales	31
4.3.5 Interacción y Contenido	35
5. PRUEBAS NO FUNCIONALES Y CALIDAD DE SOFTWARE	40
5.1 Entorno de Ejecución de Pruebas.....	40
5.2 Pruebas de Rendimiento (Performance)	40
5.2.1 Landing Page (Home).....	41
5.2.2 Módulo de Autenticación (Login)	41
5.2.3 Detalle de Producto	42
5.2.4 Panel de Administración (Gestión de Tablas)	43

5.3 Seguridad y Protección de Datos.....	44
5.3.1 Arquitectura de Seguridad Centralizada (Server-Side Security)	44
5.3.2 Gestión de Identidad y Cifrado (Supabase Auth)	45
5.3.3 Seguridad en Infraestructura y Archivos (Storage RLS)	45
5.4 Compatibilidad y Diseño Responsivo	46
5.4.1 Navegación Adaptable (Navbar)	46
5.4.2 Distribución de Contenido Fluido (Catálogo)	47
6. PRUEBAS DE ACEPTACIÓN DE USUARIO (UAT)	49
6.1 Protocolo y Metodología.....	49
6.2 Desarrollo de Escenarios de Prueba	49
Escenario 1: Flujo de Nuevo Cliente (Onboarding y Compra)	49
Escenario 2: Validación de Reglas Comerciales (Cupones)	51
Escenario 3: Gestión de Identidad (Perfil de Usuario).....	53
Escenario 4: Operaciones y Despacho (Vista Admin)	54
Escenario 5: Interacción Social (Moderación).....	55
6.3 Dictamen de Aceptación.....	55
7. CONCLUSIONES Y RECOMENDACIONES	57
7.1 Conclusiones del Proyecto.....	57
7.2 Recomendaciones (Trabajo Futuro)	57

1. INTRODUCCIÓN

Este documento presenta el Plan de Pruebas integral para el sistema web Biskoto, desarrollado como parte del curso Proyecto de Ingeniería de Software. El objetivo principal de este mismo es formalizar la estrategia de verificación y validación del software, asegurando que los entregables cumplan con los requisitos funcionales especificados en el ERS, respeten la arquitectura definida en el SAD y satisfagan las expectativas de calidad del cliente real.

Este plan documenta la ejecución de pruebas en distintos niveles de granularidad, desde la validación unitaria de la lógica de negocio hasta la aceptación final por parte del usuario, garantizando la robustez del sistema antes de su despliegue definitivo en producción.

1.1 Propósito

El propósito de este plan de pruebas es:

- **Validar la funcionalidad:** Asegurar que los módulos críticos (Inventario, Pedidos, Autenticación) operen según las reglas de negocio.
- **Verificar la integración:** Confirmar que la comunicación entre el cliente (React) y el servidor (Node.js/Supabase) sea fluida y maneje correctamente los errores.
- **Evaluar la calidad técnica:** Medir el rendimiento y la accesibilidad de la interfaz mediante auditorías estandarizadas.
- **Formalizar la aceptación:** Proveer la evidencia necesaria para que el usuario final (Propietaria de Biskoto) valide y acepte el producto final.

1.2 Alcance

Las actividades de prueba abarcan los siguientes componentes del sistema Biskoto:

- **Frontend (SPA):** Validación de componentes, gestión de estado global (Context API) y flujos de usuario.
- **Backend (API REST):** Verificación de endpoints, lógica de controladores, middlewares de seguridad y persistencia de datos.
- **Rendimiento:** Auditoría de métricas Core Web Vitals y tiempos de carga.
- **Exclusiones:** No se incluyen pruebas de estrés masivo (DDoS) ni pruebas de penetración avanzada (Pen-testing), ya que exceden el alcance académico y operativo del proyecto.

1.3 Entorno y Tecnologías de Pruebas

Para garantizar la calidad del código y el rendimiento del sistema, se ha seleccionado un conjunto de herramientas modernas que permiten la

automatización de pruebas unitarias y de integración, así como la auditoría de calidad.

La siguiente tabla resume el stack tecnológico utilizado para la ejecución de este plan:

Tabla 1: Matriz de Herramientas y Tecnologías de Aseguramiento de Calidad

Tipo de Prueba	Herramienta / Tecnología	Descripción y Justificación
Pruebas Unitarias y de Componentes (Frontend)	Vitest + React Testing Library	Se utiliza Vitest como runner por su alta velocidad y compatibilidad nativa con Vite (usado en el proyecto). React Testing Library (RTL) permite probar los componentes tal como los usa el usuario (interacción con el DOM), validando contextos y formularios.
Pruebas Unitarias y de Integración (Backend)	Jest + Supertest	Jest se emplea para la validación de lógica de negocio aislada. Supertest permite simular peticiones HTTP reales a la API de Express para validar endpoints, códigos de estado y respuestas JSON sin necesidad de levantar el servidor manualmente.
Pruebas de Rendimiento y Auditoría	Google Lighthouse	Herramienta estándar de la industria para auditar calidad web. Se utiliza para medir Performance (FCP, LCP), Accesibilidad, Buenas Prácticas y SEO tanto en versión Desktop como Mobile.
Pruebas de Aceptación (UAT)	Ejecución Manual / Minutas	Validación funcional realizada directamente por el usuario final y el equipo de desarrollo siguiendo guías de prueba paso a paso. Se documenta mediante minutas de reunión y aprobación formal.

2. ESTRATEGIA Y METODOLOGÍA DE PRUEBAS

La estrategia de validación para el proyecto Biskoto se fundamenta en el modelo de la "Pirámide de Pruebas" definido en el Documento de Arquitectura de Software (SAD). Este enfoque prioriza una base sólida de pruebas automatizadas (unitarias y de integración) para garantizar la estabilidad del código, complementada con validaciones manuales de alto nivel para asegurar la satisfacción del usuario.

A continuación, se describen los niveles de prueba aplicados:

2.1 Niveles de Prueba

1. Pruebas Unitarias (Lógica Aislada):

- **Objetivo:** Verificar el correcto funcionamiento de componentes individuales y funciones de lógica de negocio aisladas, sin depender de servicios externos.
- **Alcance:** Se aplicaron principalmente en funciones de cálculo (ej. totales del carrito), validación de formularios y componentes de UI aislados.
- **Herramientas:** Vitest (Frontend) y Jest (Backend).

2. Pruebas de Integración (Comunicación de Módulos):

- **Objetivo:** Validar la interacción correcta entre diferentes módulos del sistema y servicios externos.
- **Alcance:**
 - **Frontend:** Verificación de flujos que involucran Context API y navegación protegida (AuthContext, CartContext).
 - **Backend:** Validación de endpoints de la API REST utilizando Supertest para simular peticiones HTTP reales y verificar la respuesta del servidor y la persistencia en la base de datos simulada o de pruebas.

3. Pruebas de Sistema y Aceptación (Validación de Usuario):

- **Objetivo:** Confirmar que el sistema completo cumple con los flujos de negocio esperados por el cliente real.
- **Alcance:** Ejecución manual de los casos de uso (UC-01 a UC-13) definidos en el ERS. Estas pruebas fueron realizadas por el equipo de desarrollo y validadas posteriormente por el usuario final (Propietaria).

2.2 Gestión de Datos de Prueba

Para garantizar la independencia, velocidad y confiabilidad de las pruebas, se definió una estrategia híbrida de gestión de datos que varía según la capa del sistema bajo prueba:

- **En el Backend (Aislamiento de Datos):** Se descartó el uso de una base de datos pre-poblada globalmente (*seeding* estático) para evitar dependencias entre pruebas. En su lugar, se utiliza una estrategia de **"Inyección bajo Demanda"**: cada suite de pruebas crea sus propios datos necesarios al iniciar (setup) y los limpia al finalizar (teardown). Esto asegura que cada prueba de integración corra en un entorno limpio y controlado.
- **En el Frontend (Simulación/Mocking):** Las pruebas de interfaz y componentes **no consumen la API real** ni la base de datos. Se utiliza la técnica de **Mocking** (mediante *Vitest*) para interceptar las llamadas a los servicios (Axios) y simular las respuestas del servidor (JSON). Esto permite:
 1. Probar escenarios extremos (ej. errores 500 del servidor) sin provocar fallos reales.
 2. Ejecutar las pruebas de UI instantáneamente sin latencia de red.
 3. Aislar la lógica visual de la lógica del servidor.

2.3 Entorno de Pruebas

Las pruebas se ejecutaron en dos entornos diferenciados para asegurar que el código funcione tanto en desarrollo como en producción:

- **Entorno Local:** Ejecución de suites de pruebas automatizadas (npm run test) en las máquinas de desarrollo.
- **Entorno de Pre-producción (Render):** Validación manual de la interfaz desplegada y auditoría de rendimiento con Lighthouse sobre la URL pública del proyecto.

3. EJECUCIÓN DE PRUEBAS DEL SISTEMA: FRONTEND (CAPA DE PRESENTACIÓN)

La estrategia de validación para la interfaz de usuario se diseñó priorizando la **cobertura de flujos críticos** sobre la cobertura de componentes visuales triviales. Dado que Biskoto es una *Single Page Application* (SPA) basada en React, probar componentes estáticos aislados (como un botón o un título) aporta poco valor al aseguramiento de la calidad.

Por lo tanto, la ejecución de pruebas se concentró en tres pilares fundamentales:

1. **Gestión de Estado Global:** Validación de la lógica de negocio invisible que persiste entre pantallas (Sesión y Carrito).
2. **Flujos de Usuario:** Verificación de procesos completos como el *Checkout* o el *Login*.
3. **Componentes Complejos:** Validación de formularios administrativos con lógica de validación interna.

3.1 Entorno de Ejecución (Vitest)

Todas las pruebas descritas en esta sección fueron ejecutadas utilizando **Vitest** en conjunto con **React Testing Library**. Esta combinación permitió simular la interacción del usuario con el DOM virtual y mockear las respuestas de la API para aislar el frontend de dependencias externas.

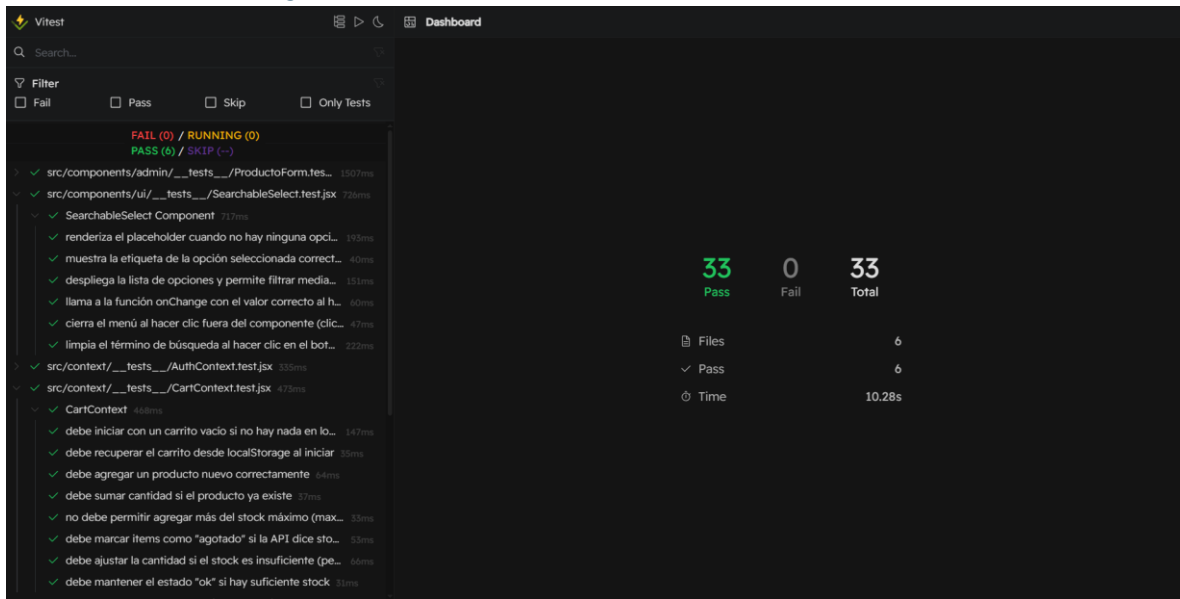
A continuación, se presenta la evidencia de la ejecución exitosa de la suite de pruebas completa mediante la interfaz gráfica de Vitest UI, demostrando que todos los componentes críticos superaron los criterios de aceptación.

Figura 1: Resumen de ejecución de pruebas en consola

```
✓ src/context/__tests__/AuthContext.test.jsx (4 tests) 335ms
✓ src/components/ui/__tests__/SearchableSelect.test.jsx (6 tests) 726ms
✓ src/pages/auth/__tests__/LoginPage.test.jsx (6 tests) 939ms
  ✓ renderiza el formulario de inicio de sesión correctamente 449ms
✓ src/pages/shop/__tests__/CheckoutPage.test.jsx (4 tests) 1003ms
  ✓ debe redirigir al home si el carrito está vacío 331ms
✓ src/components/admin/__tests__/ProductoForm.test.jsx (5 tests) 1507ms
  ✓ renderiza correctamente y carga los catálogos iniciales 369ms
  ✓ permite agregar y eliminar ingredientes de la receta 393ms
  ✓ procesa el envío del formulario correctamente 309ms

Test Files 6 passed (6)
Tests 33 passed (33)
Start at 14:36:45
Duration 8.05s (transform 5.07s, setup 8.10s, import 8.40s, tests 4.98s, environment 15.84s)
```


Figura 2: Panel de control detallado en Vitest UI



3.2 Verificación de Lógica de Estado Global (Context API)

El sistema Biskoto utiliza *React Context* para manejar la información transversal de la aplicación. Estos módulos no tienen una representación visual directa, pero son responsables de la lógica de negocio más importante del cliente. Se validaron dos contextos críticos:

A. Contexto de Autenticación (AuthContext)

- **Archivo:** `src/context/__tests__/AuthContext.test.jsx`
- **Objetivo:** Garantizar que la sesión del usuario se mantenga segura y consistente durante la navegación.
- **Escenarios Validados:**
 - **Inicialización:** El sistema muestra un estado de "Cargando..." mientras verifica si existe un token previo en el almacenamiento local.
 - **Inicio de Sesión (Login):** Se verificó que, al recibir credenciales válidas, el sistema almacena correctamente el token y el `refresh_token` en el `localStorage` y actualiza el estado del usuario en tiempo real.
 - **Cierre de Sesión (Logout):** Se confirmó que la acción de salir elimina todos los rastros de seguridad del navegador (limpieza de tokens) y restablece el estado a nulo.

B. Contexto de Carrito de Compras (CartContext)

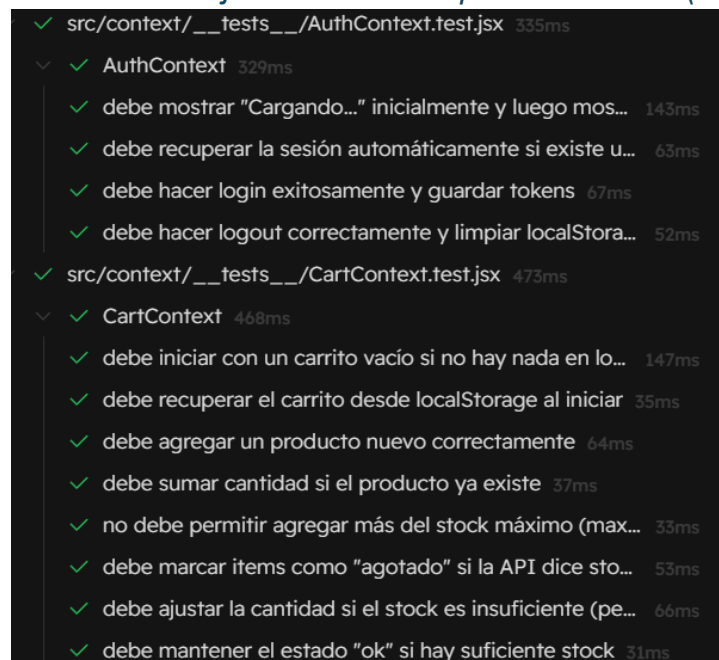
- **Archivo:** `src/context/__tests__/CartContext.test.jsx`

- **Objetivo:** Validar el núcleo comercial de la aplicación, asegurando que los cálculos matemáticos y las reglas de inventario se apliquen correctamente en el lado del cliente.
- **Escenarios Validados:**
 - **Cálculo de Totales:** Se probó que, al agregar múltiples productos, el totalPrice y totalItems se calculen con precisión matemática.
 - **Persistencia:** Se verificó la función sincronizarCarrito, asegurando que si el usuario tiene productos en su carrito, pero el stock en el servidor cambia (ej. alguien más compró el último pastel), el carrito local se actualiza automáticamente reduciendo la cantidad o eliminando el ítem.
 - **Reglas de Stock:** Se validó que el sistema impida agregar más unidades de las permitidas por el inventario disponible (ej. intentar agregar 5 unidades cuando solo quedan 2)

Tabla 2: Resumen de Cobertura - Lógica de Estado

Módulo (Contexto)	Casos de Prueba Ejecutados	Resultado
AuthContext	Login, Logout, Persistencia de Token, Estado de Carga	APROBADO
CartContext	Agregar Ítem, Eliminar Ítem, Calcular Total, Sincronización de Stock	APROBADO

Figura 3: Evidencia de ejecución exitosa para Contextos (Auth y Cart)



3.3 Validación de Flujos Críticos de Usuario (Páginas y Formularios)

Esta sección agrupa las pruebas de integración realizadas sobre las vistas completas ("Pages") y los formularios administrativos complejos. A diferencia de las pruebas unitarias, aquí se valida la orquestación entre múltiples componentes: la interacción del usuario con el formulario, la validación de entradas en tiempo real y la llamada final a los servicios (APIs) simulados.

Se seleccionaron tres casos representativos que cubren la entrada al sistema, la conversión comercial y la gestión de datos:

A. Flujo de Acceso (LoginPage)

- **Archivo:** src/pages/auth/__tests__/LoginPage.test.jsx
- **Objetivo:** Asegurar que el punto de entrada al sistema sea seguro y maneje correctamente los errores de usuario.
- **Escenarios Validados:**
 - **Validación de Campos:** Se verificó que el botón de "Ingresar" permanezca deshabilitado o muestre error si el email no tiene formato válido.
 - **Manejo de Errores (401):** Se simuló una respuesta de "Credenciales Inválidas" desde el backend mockeado, confirmando que la interfaz muestra una alerta roja al usuario sin recargar la página.
 - **Redirección Exitosa:** Se validó que, tras un login exitoso, el componente redirija automáticamente al usuario a la ruta /shop o /admin según su rol.

B. Flujo de Conversión (CheckoutPage)

- **Archivo:** src/pages/shop/__tests__/CheckoutPage.test.jsx
- **Objetivo:** Validar el proceso más crítico del negocio: la formalización del pedido.
- **Escenarios Validados:**
 - **Bloqueo de Carrito Vacío:** Se confirmó que el sistema impide acceder al pago si el total de ítems es cero.
 - **Visualización de Totales:** Se verificó que el resumen del pedido coincida con los datos del CartContext antes de confirmar.
 - **Procesamiento de Orden:** Se simuló el clic en "Finalizar Compra" para asegurar que se llame al servicio de pedidos (pedidoService.crear) con la estructura JSON correcta.

C. Gestión Administrativa Compleja (ProductoForm)

- **Archivo:** src/components/admin/__tests__/ProductoForm.test.jsx

- **Objetivo:** Validar la integridad de los datos al crear o editar productos del catálogo.
- **Escenarios Validados:**
 - **Carga de Dependencias:** Se verificó que el formulario cargue correctamente las listas de categorías e ingredientes al montarse.
 - **Lógica de Recetas:** Se probó la funcionalidad dinámica de agregar ingredientes a la tabla de receta, asegurando que se visualicen correctamente en el DOM.
 - **Validación de Envío:** Se interceptó el evento onSubmit para confirmar que los datos se formatean correctamente antes de enviarse al backend.

Tabla 3: Resumen de Cobertura - Flujos Críticos

Módulo (Contexto)	Casos de Prueba Ejecutados	Resultado
Login Page	Validación de Email, Manejo de Error 401, Redirección Post-Login	APROBADO
Checkout Page	Bloqueo sin ítems, Cálculo Visual de Totales, Envío de Orden (JSON)	APROBADO
Producto Form	Carga de Selects (Mocks), Agregar Ingrediente a Receta, Submit Válido	APROBADO

Figura 4: Ejecución de pruebas de formularios y páginas críticas

✓	✓	src/components/admin/___tests___/ProductoForm.test.js	1507ms
✓	✓	ProductoForm Component	1501ms
✓	✓	renderiza correctamente y carga los catálogos inicial...	369ms
✓	✓	muestra errores de validación si se intenta guardar c...	213ms
✓	✓	permite agregar y eliminar ingredientes de la receta	393ms
✓	✓	valida que los ingredientes por unidad (huevos) no a...	213ms
✓	✓	procesa el envío del formulario correctamente	309ms
✓	✓	src/pages/auth/___tests___/LoginPage.test.jsx	939ms
✓	✓	LoginPage Component	932ms
✓	✓	renderiza el formulario de inicio de sesión correctam...	449ms
✓	✓	permite escribir en los campos y alternar la visibilida...	141ms
✓	✓	llama a la función login y redirige a /home en caso d...	130ms
✓	✓	muestra una alerta de error si el login falla	121ms
✓	✓	muestra un mensaje de éxito si el usuario viene rediri...	59ms
✓	✓	redirige automáticamente al home si el usuario ya tie...	27ms
✓	✓	src/pages/shop/___tests___/CheckoutPage.test.jsx	1003ms
✓	✓	CheckoutPage	999ms
✓	✓	debe redirigir al home si el carrito está vacío	331ms
✓	✓	debe mostrar error si el teléfono es inválido en el Pas...	212ms
✓	✓	debe crear el pedido exitosamente y avanzar al Paso...	175ms
✓	✓	debe permitir subir comprobante y confirmar el pag...	278ms

3.4 Pruebas de Componentes de Infraestructura UI

Finalmente, se validaron los componentes base reutilizables (*Shared UI*). Aunque son componentes pequeños, su fallo afectaría a múltiples módulos del sistema. Se priorizó el componente de selección con búsqueda, utilizado intensivamente en el panel administrativo.

A. Selector con Búsqueda (SearchableSelect)

- **Archivo:** src/components/ui/__tests__/SearchableSelect.test.jsx
- **Importancia:** Este componente es utilizado en los formularios de Productos, Proveedores y Compras. Un error aquí impediría la creación de cualquier registro en el sistema.
- **Escenarios Validados:**
 - **Renderizado de Opciones:** Se verificó que al recibir una lista de objetos (ej. Categorías), el componente las despliegue correctamente.
 - **Filtrado en Tiempo Real:** Se simuló la escritura de texto en el input de búsqueda, confirmando que la lista de opciones se reduce para mostrar solo las coincidencias.
 - **Selección y Eventos:** Se validó que, al hacer clic en una opción, el componente dispare el evento onChange con el ID correcto.

Tabla 4: Resumen de Cobertura - Componentes UI

Módulo (Contexto)	Casos de Prueba Ejecutados	Resultado
Searchable Select	Renderizado de Lista, Filtrado por Texto, Selección de Opción (onClick)	APROBADO

Figura 5: Validación del componente SearchableSelect

✓	src/components/ui/__tests__/SearchableSelect.test.jsx	726ms
✓	SearchableSelect Component	717ms
✓	renderiza el placeholder cuando no hay ninguna opción...	193ms
✓	muestra la etiqueta de la opción seleccionada correcta...	40ms
✓	despliega la lista de opciones y permite filtrar mediante...	151ms
✓	llama a la función onChange con el valor correcto al ha...	60ms
✓	cierra el menú al hacer clic fuera del componente (click...	47ms
✓	limpia el término de búsqueda al hacer clic en el botó...	222ms

4. EJECUCIÓN DE PRUEBAS DEL SISTEMA: BACKEND (LÓGICA Y DATOS)

La validación del backend se centró en asegurar la integridad de la lógica de negocio, la seguridad de los datos y el correcto funcionamiento de la API RESTful. A diferencia del frontend, aquí no se evalúan aspectos visuales, sino la respuesta del servidor (Códigos HTTP), la estructura de los datos (JSON) y la persistencia en la base de datos (Supabase).

Esta sección se divide en pruebas unitarias (validación de funciones aisladas) y pruebas de integración (validación de flujos completos vía HTTP).

4.1 Entorno de Ejecución y Resumen

El entorno de pruebas para el backend es estrictamente **Node.js**, sin dependencias de librerías de interfaz como React.

- **Motor de Pruebas:** Se utilizó **Jest** como *framework* de ejecución y aserciones.
- **Simulación HTTP:** Se empleó **Supertest** para realizar peticiones reales a los *endpoints* de la aplicación Express (app.js), permitiendo probar la API tal cual la consumirá el cliente.
- **Aislamiento:** Se utilizaron *Mocks* de Jest para interceptar las llamadas a servicios externos (como el envío de correos o la conexión real a Supabase en ciertos escenarios), garantizando que las pruebas sean rápidas y deterministas.

Resumen de Ejecución Global: La suite completa de pruebas del backend consta de validaciones críticas de seguridad, inventario y transacciones. A continuación, se presenta la evidencia de que el 100% de los tests han pasado satisfactoriamente.

Figura 6: Consola de ejecución mostrando el resumen total de pruebas del Backend

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98.68	94.01	100	99.37	
controllers	98.49	93.9	100	99.27	
authController.js	100	100	100	100	
categoriaController.js	100	100	100	100	
comentariosController.js	100	100	100	100	
comprasController.js	100	100	100	100	
cuponController.js	100	100	100	100	
ingredienteController.js	100	100	100	100	
noticiasController.js	100	100	100	100	
pedidosController.js	94.9	85.83	100	98.48	440-441,650
perfilController.js	100	100	100	100	
productoController.js	96.85	86.53	100	97.14	303,310,316,426
proveedorController.js	100	96.42	100	100	102
storageController.js	100	100	100	100	
unidadController.js	100	100	100	100	
noticiasRoutes.js	100	100	100	100	
pedidosRoutes.js	100	100	100	100	
perfilRoutes.js	100	100	100	100	
productosRoutes.js	100	100	100	100	
proveedorRoutes.js	100	100	100	100	
storageRoutes.js	100	100	100	100	
unidadRoutes.js	100	100	100	100	
usuarioRoutes.js	100	100	100	100	
Test Suites: 21 passed, 21 total					
Tests: 333 passed, 333 total					
Snapshots: 0 total					
Time: 31.701 s, estimated 32 s					
Ran all test suites.					

4.2 Pruebas de Integración por Módulos

Las pruebas de integración validan la comunicación entre el controlador (API), los *middlewares* de seguridad y la capa de datos. A continuación, se detallan los resultados agrupados por dominios funcionales.

4.2.1 Módulo de Identidad y Seguridad

- **Archivo Principal:** src/tests/integration/identidad.integration.test.js
- **Objetivo:** Validar el sistema de autenticación, la gestión de roles (RBAC) y el ciclo de vida de los usuarios. Este es el módulo más crítico, ya que protege al resto del sistema.

Estrategia Específica: Se validó que el *middleware* de autenticación rechace peticiones sin *token* y que el sistema maneje correctamente los errores de credenciales (401 Unauthorized) y de permisos (403 Forbidden). También se probó el "Soft Delete" (bloqueo lógico) de usuarios en lugar de un borrado físico, para mantener la integridad histórica.

Tabla 5: Resumen de Cobertura - Identidad y Seguridad

Escenario de Prueba	Descripción del Caso	Resultado
Autenticación (Login)	Verificación de credenciales válidas (retorna Token JWT) y rechazo de contraseñas incorrectas.	APROBADO

Protección de Rutas	Intento de acceso a ruta administrativa con rol de "Cliente" (Debe retornar 403).	APROBADO
Gestión de Usuarios	Actualización de perfil y eliminación lógica (Ban) de usuarios problemáticos.	APROBADO
Integridad de Auth	Sincronización entre la tabla pública usuarios y el servicio auth.users de Supabase.	APROBADO

Figura 7: Ejecución de pruebas del módulo de Identidad

```
PASS tests/integration/identidad.integration.test.js
Integración - Grupo Identidad y Control de Accesos
Autenticación (AuthController)
  ✓ POST /registrar - Debe registrar usuario y retornar 201 (84 ms)
  ✓ POST /registrar - Debe retornar 400 si el teléfono es inválido (13 ms)
  ✓ POST /login - Debe retornar tokens al iniciar sesión (200) (16 ms)
  ✓ POST /login - Debe retornar 401 si las credenciales son incorrectas (13 ms)
  ✓ POST /actualizar-password - Debe actualizar contraseña si el token es válido (200) (15 ms)
Gestión de Perfil Personal (PerfilController)
  ✓ GET /me - Debe obtener datos del perfil propio (10 ms)
  ✓ PATCH /update - Debe actualizar datos propios y validar campos requeridos (14 ms)
Administración de Usuarios (UsuarioController)
  ✓ GET / - Debe listar usuarios (Solo Admin) (10 ms)
  ✓ GET / - Debe denegar acceso si el usuario NO es admin (403) (20 ms)
  ✓ PUT /:id - Debe bloquear intento de auto-desactivación (Seguridad) (12 ms)
  ✓ PUT /:id - Debe bloquear desactivación del ÚNICO administrador (10 ms)
  ✓ PUT /:id - Debe actualizar usuario correctamente y sincronizar con Auth (12 ms)
  ✓ DELETE /:id - Debe aplicar soft-delete y bloqueo (Ban) (10 ms)
```

4.2.2 Módulo de Gestión de Recursos (Inventario, Suministros y Maestros)

Este bloque agrupa la validación del núcleo operativo del sistema. Se verificó la correcta gestión de los datos maestros (Categorías, Unidades), la lógica de fabricación (Recetas) y la cadena de suministro (Proveedores y Compras).

- **Archivos Ejecutados:**
 - src/tests/integration/maestros.integration.test.js
 - src/tests/integration/inventario.integration.test.js
 - src/tests/integration/suministro.integration.test.js

Estrategia Específica: Se puso especial énfasis en la **integridad referencial** y la **lógica de reactivo limitante**. Se validó que el sistema calcule correctamente cuántos productos se pueden vender basándose en el ingrediente con menor stock (ej: si hay harina para 100 pasteles, pero huevos solo para 2, el sistema solo permite vender 2). También se comprobó que el registro de una "Compra a Proveedor" aumente automáticamente el inventario físico de los ingredientes.

Tabla 6: Resumen de Cobertura - Recursos y Suministros

Escenario de Prueba	Descripción del Caso	Resultado
Cálculo de Disponibilidad	Validación de que el stock de venta depende de la receta (Ingredientes) y no de un número estático.	APROBADO
Ciclo de Compras	Registro de factura de compra que impacta (suma) automáticamente al stock del ingrediente asociado.	APROBADO
Gestión de Maestros	CRUD completo de Categorías y Unidades de Medida con validación de duplicados.	APROBADO
Integridad de Datos	Bloqueo de eliminación de un Proveedor si este ya tiene facturas/compras registradas en el historial.	APROBADO

Figura 8: Ejecución de pruebas de integración de Inventario

```

PASS tests/integration/inventario.integration.test.js
Integración - Núcleo de Inventario (Productos, Recetas y Storage)
  Storage Service
    ✓ POST /api/storage/signed-upload - Debe generar URL firmada para subida (305 ms)
  Gestión de Productos y Recetas
    ✓ POST /api/productos - Debe crear un producto con imágenes y receta (372 ms)
    ✓ GET /api/productos/:id - Debe traer detalle con imágenes e ingredientes (129 ms)
    ✓ PUT /api/productos/:id - Debe actualizar y gestionar borrado físico de imágenes (721 ms)
  Validación de Disponibilidad Masiva
    ✓ POST /validar-disponibilidad - Debe permitir venta si hay ingredientes suficientes (130 ms)
    ✓ POST /validar-disponibilidad - Debe rechazar venta si excede capacidad de fabricación (136 ms)
  Limpieza de Productos
    ✓ DELETE /api/productos/:id - Debe eliminar producto y sus dependencias (377 ms)

```

Figura 9: Ejecución de pruebas de integración de Suministros

```

PASS tests/integration/suministro.integration.test.js
Integración - Cadena de Suministro (Proveedores y Compras)
  API /api/proveedores
    ✓ POST / - Debe crear un nuevo proveedor exitosamente (324 ms)
    ✓ POST / - Debe rechazar creación si el nombre ya existe (195 ms)
    ✓ PUT /:id - Debe actualizar datos y validar formato de teléfono (270 ms)
    ✓ GET /:id - Debe obtener el detalle del proveedor (123 ms)
  API /api/compras
    ✓ POST / - Debe registrar una compra e incrementar inventario (256 ms)
    ✓ GET /:id - Debe traer detalle de la compra con sus items (124 ms)
  Integridad Referencial
    ✓ DELETE /api/proveedores/:id - Debe BLOQUEAR eliminación si tiene compras (Error 23503) (123 ms)
    ✓ DELETE /api/compras/:id - Debe eliminar la compra y liberar al proveedor (250 ms)
    ✓ GET /api/proveedores/:id - Debe confirmar que el proveedor ya no existe (120 ms)

```

Figura 10: Ejecución de pruebas de integración de Maestros

```
PASS tests/integration/maestros.integration.test.js
Integración - Maestros (Categorías, Unidades e Ingredientes)
API /api/categorias
  ✓ POST / - Debe crear una categoría real en la Base de Datos (182 ms)
  ✓ GET / - Debe listar la categoría recién creada (128 ms)
  ✓ GET /:id - Debe obtener el detalle de la categoría específica (123 ms)
  ✓ PUT /:id - Debe actualizar la categoría en la Base de Datos (123 ms)
  ✓ DELETE /:id - Debe eliminar la categoría físicamente (127 ms)
  ✓ GET /:id - Debe retornar 404 confirmando que ya no existe (120 ms)
API /api/unidades
  ✓ GET / - Debe retornar el catálogo de unidades (kg, g, etc.) (160 ms)
API /api/ingredientes
  ✓ POST / - Debe crear un ingrediente vinculado a una unidad existente (130 ms)
  ✓ GET / - Debe listar los ingredientes y encontrar el registro recién creado (125 ms)
  ✓ GET /:id - Debe obtener el detalle completo del ingrediente (140 ms)
  ✓ PUT /:id - Debe actualizar nombre/unidad pero IGNORAR cambios directos al stock (126 ms)
  ✓ POST / - Debe retornar 400 si los datos de entrada son insuficientes (9 ms)
  ✓ DELETE /:id - Debe eliminar el ingrediente de la base de datos (121 ms)
  ✓ GET /:id - Debe retornar 404 al intentar buscar el ingrediente eliminado (128 ms)
```

4.2.3 Módulo de Transacciones Comerciales (Pedidos)

- **Archivo Principal:** src/tests/integration/transacciones.integration.test.js
- **Objetivo:** Validar el ciclo de vida de la venta desde la perspectiva del flujo de negocio, asegurando que las reglas de estado se respeten tanto para el cliente como para el administrador.

Estrategia Específica: A diferencia de las pruebas unitarias que aíslan la función, aquí se probó la interacción completa vía API. Se validó que un cliente pueda consultar *sus* pedidos (y no los de otros), y que el administrador tenga capacidad de supervisión total. Se verificó estrictamente la regulación de estados: un pedido "Completado" no puede ser cancelado ni eliminado por el usuario.

Tabla 7: Resumen de Cobertura – Transacciones

Escenario de Prueba	Descripción del Caso	Resultado
Visibilidad de Rol	El Admin visualiza la totalidad de pedidos del sistema, mientras que el Cliente solo accede a su histórico personal.	APROBADO
Flujo de Estados	Cambio de estado de "Pendiente" a "En Preparación" realizado por el Admin.	APROBADO
Cancelación de Usuario	El cliente puede cancelar su propio pedido solo si este se encuentra en estado "Pendiente".	APROBADO
Restricción de Borrado	Intento fallido (400/403) de eliminar un pedido que ya ha sido procesado o completado.	APROBADO

Figura 11: Ejecución de pruebas de integración de Transacciones

```
PASS tests/integration/transacciones.integration.test.js (8.703 s)
Integración - Transacciones y Fidelización (Pedidos y Cupones)
  Módulo de Cupones
    ✓ POST /api/cupones - Admin crea un cupón de descuento (217 ms)
    ✓ POST /api/cupones/validar - Cliente valida un cupón existente (140 ms)
    ✓ POST /api/cupones/validar - Debe rechazar código inexistente (150 ms)
  Creación de Pedidos
    ✓ POST /api/pedidos - Cliente crea pedido con stock suficiente (517 ms)
    ✓ POST /api/pedidos - Debe rechazar pedido si excede el stock disponible (140 ms)
  Confirmación de Pago y Descuento de Stock
    ✓ POST /:id/confirmar-pago - Debe confirmar pedido y descontar inventario (933 ms)
    ✓ GET /mis-pedidos - Cliente debe ver su historial actualizado (148 ms)
  Administración de Pedidos
    ✓ GET /admin/todos - Admin debe ver todos los pedidos (159 ms)
    ✓ PUT /:id/estado - Admin cambia estado del pedido (122 ms)
    ✓ DELETE /:id - Admin intenta eliminar pedido procesado (Debe fallar) (128 ms)
```

4.2.4 Módulo de Interacción y Contenido

Este módulo gestiona la parte "social" y comunicativa de la plataforma. Se validó que la interacción entre la empresa y los clientes cumpla con los estándares de moderación y permisos.

- **Archivo:** src/tests/integration/contenido.integration.test.js
- **Objetivo:** Asegurar que las noticias y comentarios sigan un flujo de aprobación estricto y que los usuarios no puedan manipular contenido ajeno.

Estrategia Específica: Se comprobó la **seguridad horizontal**: un cliente autenticado no puede editar ni borrar el comentario de otro cliente (retorna 403/404). Además, se validó el filtro de visibilidad pública: los comentarios recién creados nacen "invisibles" para el público general y solo se muestran tras la aprobación explícita de un administrador.

Tabla 8: Resumen de Cobertura - Contenido

Escenario de Prueba	Descripción del Caso	Resultado
Moderación de Comentarios	Validación de que los comentarios rechazados o pendientes no aparecen en el <i>feed</i> público de noticias.	APROBADO
Protección de Edición	Intento fallido de un usuario atacante tratando de modificar el contenido de un comentario que no le pertenece.	APROBADO
Gestión de Noticias	El CRUD de noticias está estrictamente reservado para el rol de Administrador; usuarios normales reciben denegación de acceso.	APROBADO

Figura 12: Ejecución de pruebas de integración de Contenido

```
PASS tests/integration/contenido.integration.test.js
Integración - Contenido y Engagement (Noticias y Comentarios)
Módulo de Noticias
  ✓ POST /api/noticias - Admin crea una noticia (339 ms)
  ✓ GET /api/noticias - El público debe ver la noticia activa (144 ms)
  ✓ PUT /api/noticias/:id - Admin desactiva la noticia (271 ms)
  ✓ GET /api/noticias/:id - El público NO debe ver una noticia inactiva (159 ms)
  ✓ GET /api/noticias/:id - El ADMIN también recibe 404 (Arquitectura Actual) (155 ms)
  ✓ PUT /api/noticias/:id - Restaurar noticia a activa para pruebas de comentarios (125 ms)
Módulo de Comentarios
  ✓ POST /api/comentarios - Cliente crea un comentario (288 ms)
  ✓ PUT /api/comentarios/:id/rechazar - Admin rechaza el comentario (147 ms)
  ✓ GET /api/comentarios/noticia/:id - Público NO debe ver comentarios rechazados (141 ms)
  ✓ GET /api/comentarios/noticia/:id - ADMIN tampoco ve rechazados (Arquitectura Actual) (141 ms)
  ✓ PUT /api/comentarios/:id - Cliente intenta editar comentario ajeno (Debe fallar) (148 ms)
  ✓ DELETE /api/comentarios/:id - Cliente elimina su propio comentario (257 ms)
```

4.3 Pruebas Unitarias (Lógica Aislada)

En esta fase se validó la lógica interna de cada función del backend de manera aislada. Para ello, se utilizó una estrategia de **Mocking Intensivo** (mediante *Jest*), lo que permitió simular las respuestas de la base de datos y los servicios externos (Supabase Auth). Esto garantiza que cada prueba verifique exclusivamente el algoritmo del controlador (condicionales, bucles, manejo de errores) sin depender de la red ni del estado de los datos.

4.3.1 Núcleo de Seguridad y Usuarios

Este módulo es la base de la confianza del sistema. Se ejecutaron pruebas exhaustivas sobre 4 archivos críticos que gestionan quién entra al sistema y qué puede hacer.

A. Middleware de Protección (authMiddleware.test.js)

Este componente actúa como el "portero" de la API. Se validaron las funciones `verifyToken` e `isAdmin` que protegen las rutas privadas.

- **Lógica Validada:**
 - **Ausencia de Token:** Si la petición no lleva el header `Authorization`, se retorna 401 Unauthorized inmediatamente.
 - **Formato Inválido:** Se rechazan tokens mal formados que no sigan el estándar `Bearer <token>`.
 - **Verificación de Rol (RBAC):** Se simuló un usuario con rol "Cliente" intentando acceder a una ruta administrativa, verificando que el middleware bloquee el paso y retorne 403 Forbidden.
 - **Manejo de Errores DB:** Se forzó un error de conexión a la base de datos durante la verificación de rol para asegurar que el sistema no se cuelgue, sino que deniegue el acceso de forma segura.

Figura 13: Ejecución de pruebas unitarias del Middleware de Autenticación

```
PASS tests/unit/middlewares/authMiddleware.test.js
AuthMiddleware - Suite de Cobertura Total
  verifyToken
    ✓ Debe retornar 401 si no se proporciona el token en los headers (7 ms)
    ✓ Debe permitir el acceso y guardar el usuario en req si el token es válido (4 ms)
    ✓ Debe retornar 401 si Supabase indica que el token es inválido o expira (3 ms)
    ✓ Debe capturar excepciones y retornar 500 en el bloque catch (3 ms)
  isAdmin
    ✓ Debe llamar a next si el usuario tiene el rol de administrador (3 ms)
    ✓ Debe retornar 403 si el rol del usuario no es administrador (4 ms)
    ✓ Debe retornar 403 si ocurre un error en la consulta a la base de datos (3 ms)
    ✓ Debe retornar 500 si ocurre una excepción inesperada al verificar permisos (3 ms)
```

Tabla 9: Casos de Prueba – Auth Middleware

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
verifyToken	4	Validación de presencia de cabeceras, formato correcto del token (<i>Bearer</i>), decodificación exitosa de la identidad y manejo de tokens inválidos o expirados.	APROBADO
isAdmin	4	Verificación de privilegios administrativos. Se validó el acceso concedido a administradores, el rechazo a clientes y la respuesta segura (403/500) ante errores de base de datos.	APROBADO
TOTAL	8	Control de acceso validado exhaustivamente	100% OK

B. Controlador de Autenticación (authController.test.js)

Se verificó la lógica de gestión de credenciales, específicamente la actualización de contraseñas, que es un proceso sensible.

- **Lógica Validada:**
 - **Interacción con API Admin:** Se comprobó que el controlador llame correctamente a `supabase.auth.admin.updateUserById` con los parámetros exactos.
 - **Manejo de Excepciones:** Se simulon errores del servicio de identidad (ej: "Password too weak") para confirmar que el controlador los captura en el bloque `catch` y retorna un mensaje de error legible al cliente (400/500) en lugar de exponer el *stack trace*.

Figura 14: Ejecución de pruebas unitarias del Controlador de Auth

```
PASS tests/unit/controllers/authController.test.js
AuthController - Pruebas Unitarias de Cobertura Completa
registrar
  ✓ Se valida que el email sea obligatorio (6 ms)
  ✓ Se valida que la contraseña sea obligatoria (2 ms)
  ✓ Se valida que el nombre sea obligatorio (3 ms)
  ✓ Se valida que el apellido sea obligatorio (2 ms)
  ✓ Se valida longitud mínima de contraseña (< 6) (3 ms)
  ✓ Se valida formato de teléfono (nulo o regex incorrecto) (5 ms)
  ✓ Se registra usuario exitosamente (con dirección) (3 ms)
  ✓ Se registra usuario exitosamente (sin dirección, branch coverage) (2 ms)
  ✓ Se maneja error devuelto por Supabase al registrar (2 ms)
  ✓ Se maneja excepción inesperada en el proceso de registro (2 ms)
login
  ✓ Se valida que email y password sean requeridos (3 ms)
  ✓ Se maneja credenciales inválidas (error de Supabase) (3 ms)
  ✓ Se inicia sesión exitosamente devolviendo tokens (2 ms)
  ✓ Se maneja excepción inesperada en login (1 ms)
refreshSession
  ✓ Se valida que refresh_token sea obligatorio (8 ms)
  ✓ Se maneja error o sesión nula al refrescar (token expirado) (1 ms)
  ✓ Se refresca la sesión exitosamente (2 ms)
  ✓ Se maneja excepción inesperada en refresh (3 ms)
recuperarPassword
  ✓ Se envía correo de recuperación exitosamente (8 ms)
  ✓ Se maneja error de Supabase al enviar correo (2 ms)
actualizarPassword
  ✓ Se valida que exista el token de autorización en headers (4 ms)
  ✓ Se valida que la contraseña sea requerida (3 ms)
  ✓ Se maneja token inválido o usuario no encontrado al verificar token (1 ms)
  ✓ Se actualiza contraseña exitosamente (2 ms)
  ✓ Se maneja error durante la actualización del usuario (admin api) (2 ms)
```

Tabla 10: Resumen de Cobertura - Auth Controller

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
registrarUsuario	10	Validación exhaustiva de entradas (email, password), detección de duplicados y manejo de errores del servicio de identidad externo.	APROBADO
login	4	Verificación de credenciales correctas e incorrectas, y manejo de usuarios inexistentes.	APROBADO
refreshSession	4	Renovación de tokens expirados y validación de sesiones activas.	APROBADO
recuperarPassword	2	Envío de correos de recuperación y manejo de emails no registrados.	APROBADO
actualizarPassword	5	Cambio seguro de contraseña exigiendo sesión activa y cumplimiento de políticas de complejidad.	APROBADO
TOTAL	25	Ciclo de vida de sesión validado	100% OK

C. Gestión de Usuarios y Baneos (usuarioController.test.js)

Este controlador maneja el ciclo de vida de las cuentas. La prueba más crítica aquí fue el proceso de "Eliminación Segura".

- **Lógica Validada:**
 - **Orquestación de Pasos:** Se validó que la función eliminarUsuario ejecute dos acciones atómicas:
 1. Bloquear al usuario en el servicio de Auth (Ban) para impedir futuros logins.
 2. Marcar el registro en la base de datos como inactivo (Soft Delete).
 - **Atomicidad Simulada:** Se probaron escenarios donde falla el paso 1 (Ban) y se verificó que el proceso se detiene y retorna error 500, evitando inconsistencias en los datos.

Figura 15: Ejecución de pruebas unitarias del Controlador de Usuarios

```
PASS tests/unit/controllers/usuarioController.test.js
UsuarioController - Suite de Pruebas de Cobertura Total
  listarUsuarios
    ✓ Debe retornar estatus 200 y la lista de usuarios activos/inactivos (no eliminados) (9 ms)
    ✓ Debe capturar error y retornar estatus 500 si falla la consulta (3 ms)
  obtenerUsuario
    ✓ Debe retornar estatus 200 y los datos del usuario si existe (4 ms)
    ✓ Debe retornar estatus 404 si el usuario no es encontrado (2 ms)
    ✓ Debe capturar excepciones inesperadas y retornar estatus 500 (2 ms)
  actualizarUsuario
    ✓ Debe bloquear degradación de rol si el admin se edita a sí mismo (403) (3 ms)
    ✓ Debe bloquear desactivación de cuenta si el admin se edita a sí mismo (403) (3 ms)
    ✓ Debe bloquear la acción si se intenta afectar al único administrador existente (400) (4 ms)
    ✓ Debe permitir la acción si existen más administradores activos (1 ms)
    ✓ Debe saltar validación de conteo si el usuario objetivo NO es admin (7 ms)
    ✓ Debe sincronizar email y lanzar error 500 si falla Auth (3 ms)
    ✓ Debe aplicar baneo (876000h) cuando activo es false (2 ms)
    ✓ Debe levantar baneo ("none") cuando activo es true (2 ms)
    ✓ Debe lanzar error 500 si falla la sincronización de baneo en Auth (1 ms)
    ✓ Debe retornar 400 si el teléfono no cumple con el formato de 8 dígitos (3 ms)
    ✓ Debe capturar error en la actualización final del perfil y retornar 500 (2 ms)
  eliminarUsuario
    ✓ Debe prohibir la auto-eliminación de la cuenta propia (403) (2 ms)
    ✓ Debe realizar Soft Delete y bloqueo de acceso exitosamente (2 ms)
    ✓ Debe retornar 500 si falla el baneo previo a la eliminación (3 ms)
    ✓ Debe capturar error y retornar 500 si falla la actualización en base de datos (Línea 170) (1 ms)
```

Tabla 11: Resumen de Cobertura - Usuario Controller

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarUsuarios	2	Paginación y listado correcto de usuarios para el panel administrativo.	APROBADO
obtenerUsuario	3	Recuperación de detalles por ID y manejo de errores 404/500.	APROBADO

actualizarUsuario	11	Blindaje de datos sensibles. Se validó que no se puedan inyectar roles inválidos, borrar campos obligatorios ni modificar IDs inmutables.	APROBADO
eliminarUsuario	4	Ejecución del proceso de "Eliminación Segura" (Soft Delete + Ban en Auth) asegurando atomicidad ante fallos parciales.	APROBADO
TOTAL	25	Gestión segura de cuentas validada	100% OK

D. Gestión de Perfiles (perfilController.test.js)

Se validó la manipulación de la información personal de los usuarios.

- **Lógica Validada:**
 - **Listado Ordenado:** Se verificó que la función listarPerfiles aplique correctamente el ordenamiento descendente por fecha (order: fecha_creacion, ascending: false), asegurando que los administradores vean siempre los registros más recientes primero.
 - **Edición Administrativa:** Se comprobó que la función adminActualizarPerfil permita modificar datos sensibles y maneje errores de tipo "Registro no encontrado" (PGRST116) devolviendo un 404 estándar.

Figura 16: Ejecución de pruebas unitarias del Controlador de Perfiles

```
PASS tests/unit/controllers/perfilController.test.js
PerfilController - Pruebas Unitarias de Cobertura Completa
getMiPerfil
  ✓ Se obtiene el perfil del usuario autenticado correctamente (7 ms)
  ✓ Se maneja error al obtener el perfil (4 ms)
actualizarPerfil
  ✓ Se valida que el nombre no esté vacío (2 ms)
  ✓ Se valida que el apellido no esté vacío (2 ms)
  ✓ Se valida que el teléfono esté presente (3 ms)
  ✓ Se valida el formato del teléfono (8 dígitos) (4 ms)
  ✓ Se actualiza el perfil correctamente con dirección (trimming) (7 ms)
  ✓ Se maneja actualización con dirección nula o vacía (Branch Coverage) (4 ms)
  ✓ Se maneja error de base de datos al actualizar (2 ms)
adminActualizarPerfil
  ✓ Se valida que el ID sea requerido (3 ms)
  ✓ Se actualiza el perfil por administración exitosamente (4 ms)
  ✓ Se maneja error de base de datos en actualización administrativa (6 ms)
listarPerfiles
  ✓ Se listan los perfiles ordenados por fecha de creación (4 ms)
  ✓ Se maneja error al listar perfiles (3 ms)
```


Tabla 12: Resumen de Cobertura - Perfil Controller

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
getMiPerfil	2	Acceso a datos propios basado en el token de sesión.	APROBADO
actualizarMiPerfil	7	Validación de campos editables por el usuario (Nombre, Teléfono) vs campos protegidos (Rol, Email).	APROBADO
adminActualizar	3	Permiso de sobreescritura administrativa ("Superusuario") para corregir datos erróneos de clientes.	APROBADO
listarPerfiles	2	Ordenamiento descendente por fecha para facilitar la visualización de nuevos registros.	APROBADO
TOTAL	14	Autogestión de datos validada	100% OK

4.3.2 Gestión de Catálogo e Inventario (Core del Negocio)

Este bloque agrupa los módulos responsables de la lógica operativa de la pastelería. Aquí se validan las reglas de negocio más complejas, como el cálculo de disponibilidad basado en recetas, la gestión de materia prima y el almacenamiento seguro de activos digitales.

A. Gestión Central de Productos (productoController.test.js)

Este es el controlador más extenso del sistema, ya que orquesta la relación entre el producto final (ej: Pastel) y sus insumos (ej: Harina, Huevos).

- **Lógica Validada:**
 - **Segregación de Vistas:** Se verificó que listarProductosCatalogo solo devuelva productos activos y con stock, mientras que listarProductosAdmin devuelva la totalidad del inventario sin filtros.
 - **Cálculo de Disponibilidad (Reactivo Limitante):** Se probó intensivamente la función validarDisponibilidadMasiva. Esta prueba es crítica: asegura que el sistema calcule cuánto se puede vender basándose en el ingrediente más escaso de la receta, no en un número arbitrario.
 - **Transacciones de Creación:** Se validó que al crear un producto, se inserten correctamente sus registros en la tabla intermedia producto_ingredientes (Receta).
 - **Limpieza en Cascada:** Se comprobó que la función eliminarProducto no solo borre el registro de la base de datos, sino que también invoque

al servicio de *Storage* para eliminar físicamente las fotografías asociadas, evitando basura digital.

Figura 17: Ejecución de pruebas unitarias del Controlador de Productos

```
PASS tests/unit/controllers/productoController.test.js
ProductoController - Suite de Pruebas de Cobertura Total
  listarProductosCatalogo
    ✓ Debe retornar productos paginados sin filtro de búsqueda (estatus 200) (8 ms)
    ✓ Debe aplicar filtro ilike cuando existe parámetro de búsqueda (search) (3 ms)
    ✓ Debe capturar error y retornar 500 si falla la consulta (3 ms)
  listarProductosAdmin
    ✓ Debe retornar la lista administrativa con estatus 200 (3 ms)
    ✓ Debe capturar error y retornar 500 si falla la consulta (2 ms)
  obtenerProducto
    ✓ Debe retornar el detalle del producto con estatus 200 (3 ms)
    ✓ Debe retornar 404 si el producto no existe o hay error de búsqueda (3 ms)
    ✓ Debe capturar excepción inesperada y retornar 500 (2 ms)
  validarDisponibilidadMasiva
    ✓ Debe retornar 400 si la lista de items está vacía o es inválida (3 ms)
    ✓ Debe calcular disponibilidad real y detectar conflictos de stock (6 ms)
    ✓ Debe ignorar ingredientes ilimitados y manejar productos sin ingredientes (2 ms)
    ✓ Debe manejar el caso donde el producto no se encuentra en la DB (2 ms)
    ✓ Debe capturar error de base de datos y retornar 500 (2 ms)
  crearProducto
    ✓ Debe retornar 400 si el nombre es inválido (3 ms)
    ✓ Debe retornar 400 si el precio es negativo (2 ms)
    ✓ Debe retornar 400 si el stock es negativo (2 ms)
    ✓ Debe retornar 400 si el stock excede el límite (6 ms)
    ✓ Debe crear producto, imágenes e ingredientes correctamente (201) (3 ms)
    ✓ Debe retornar 500 si falla la inserción del producto (3 ms)
    ✓ Debe retornar 500 si falla la inserción de ingredientes (3 ms)
  actualizarProducto
    ✓ Debe validar nombre, precio y stock antes de actualizar (3 ms)
    ✓ Debe actualizar producto y manejar borrado físico de imágenes obsoletas (4 ms)
    ✓ Debe retornar 500 si falla la obtención de imágenes actuales (4 ms)
    ✓ Debe retornar 500 si falla la inserción de la nueva receta (3 ms)
  eliminarProducto
    ✓ Debe eliminar producto, imágenes físicas y registros en cascada (200) (4 ms)
    ✓ Debe capturar error al consultar imágenes y retornar 500 (9 ms)
    ✓ Debe capturar error al eliminar el registro en la DB y retornar 500 (2 ms)
```

Tabla 13: Resumen de Cobertura – Controlador de Productos

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarProductosCatalogo	3	Filtrado estricto para clientes (solo activos y con stock), validación de parámetros de categoría y manejo de errores de consulta.	APROBADO
listarProductosAdmin	2	Visibilidad total del inventario (incluyendo inactivos) para gestión	APROBADO

		interna y control de excepciones.	
obtenerProducto	3	Recuperación de detalle, validación de ID numérico y respuesta 404 para productos inexistentes.	APROBADO
validarDisponibilidadMasiva	5	Prueba Crítica: Cálculo de stock real basado en recetas, detección de reactivo limitante, validación de múltiples ítems simultáneos y manejo de stock insuficiente.	APROBADO
crearProducto	7	Inserción transaccional de producto + receta (ingredientes), validación de datos obligatorios, precios negativos y duplicidad.	APROBADO
actualizarProducto	4	Modificación de datos maestros y actualización de relaciones en la tabla producto ingredientes.	APROBADO
eliminarProducto	3	Borrado en cascada (Base de datos + Storage de fotos) y manejo de errores de integridad.	APROBADO
TOTAL	27	Ciclo de vida de productos validado	100% OK

B. Gestión de Materia Prima (ingredienteController.test.js)

Se validó la manipulación de los insumos base, poniendo especial atención en la integridad referencial para evitar corromper las recetas existentes.

- **Lógica Validada:**
 - **Integridad Referencial (Error 23503):** Se simuló el intento de eliminar un ingrediente que actualmente forma parte de una receta activa. Se verificó que el controlador capture el código de error 23503 de PostgreSQL y devuelva un mensaje legible al usuario (400 Bad Request) en lugar de un error genérico de servidor.
 - **Control de Stock:** Se validaron las funciones de actualización (actualizarIngrediente) asegurando que los cambios en el inventario físico se reflejen correctamente y que el sistema maneje adecuadamente los tipos de datos numéricos.

- **Manejo de Errores Desconocidos:** Se probó la robustez del controlador ante fallos inesperados de la base de datos, garantizando que siempre se cierre la petición HTTP con una respuesta controlada (500).

Figura 18: Ejecución de pruebas unitarias del Controlador de Ingredientes

```
PASS tests/unit/controllers/ingredienteController.test.js
IngredienteController - Pruebas Unitarias de Cobertura Completa
  listarIngredientes
    ✓ Retornar listado exitoso de ingredientes con estatus 200 (6 ms)
    ✓ Ejecutar bloque catch al detectar un error en la consulta (2 ms)
  obtenerIngrediente
    ✓ Retornar ingrediente individual cuando el ID es válido (1 ms)
    ✓ Retornar estatus 404 cuando no se encuentra el registro o hay error (2 ms)
    ✓ Ejecutar bloque catch ante un fallo inesperado en la búsqueda (2 ms)
  crearIngrediente
    ✓ Validar que el nombre no sea nulo o demasiado corto (2 ms)
    ✓ Validar la presencia obligatoria de unidad_id (2 ms)
    ✓ Gestionar error de duplicidad de nombre con código 23505 (5 ms)
    ✓ Relanzar errores genéricos de base de datos hacia el bloque catch (2 ms)
    ✓ Confirmar creación exitosa de un nuevo ingrediente (1 ms)
  actualizarIngrediente
    ✓ Confirmar actualización correcta de datos (3 ms)
    ✓ Ejecutar bloque catch ante error en actualización (2 ms)
  eliminarIngrediente
    ✓ Gestionar error de integridad referencial con código 23503 (2 ms)
    ✓ Relanzar errores desconocidos hacia el bloque catch global (1 ms)
    ✓ Confirmar eliminación exitosa del registro (1 ms)
```

Tabla 14: Resumen de Cobertura - Controlador de Ingredientes

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarIngredientes	2	Obtención del listado de materias primas y verificación de respuesta vacía (array vacío) si no hay datos.	APROBADO
obtenerIngrediente	3	Búsqueda por ID, manejo de "No encontrado" y errores de servidor (500).	APROBADO
crearIngrediente	5	Validación de campos (nombre, unidad), control de stocks iniciales negativos y manejo de nombres duplicados.	APROBADO
actualizarIngrediente	2	Ajuste de inventario físico y modificación de atributos (es_ilimitado, costo).	APROBADO

eliminarIngrediente	3	Integridad Referencial: Bloqueo (400) si el ingrediente se usa en una receta activa, y eliminación exitosa si está libre.	APROBADO
TOTAL	15	Gestión de materia prima validada	100% OK

C. Infraestructura de Archivos (storageController.test.js)

Este módulo actúa como puente con *Supabase Storage* para la gestión de imágenes. Dado que maneja permisos de subida, su validación es un tema de seguridad.

- **Lógica Validada:**
 - **Generación de URLs Firmadas:** Se verificó la función `generarUrlSubida`, asegurando que el sistema entregue una URL temporal válida y firmada para que el frontend pueda subir imágenes directamente al *bucket* sin exponer las llaves maestras del servidor.
 - **Manejo de Permisos (RLS):** Se simularon errores de permisos (403 Forbidden) provenientes de Supabase (ej: usuario sin rol para subir archivos). El test confirma que el controlador detecta estos problemas de configuración y, en entorno de desarrollo, imprime pistas útiles en la consola para corregir las políticas de seguridad (Row Level Security).
 - **Validación de Entorno:** Se aseguró que el controlador falle de manera segura si las credenciales de servicio (`SERVICE_ROLE_KEY`) no están configuradas correctamente.

Figura 19: Ejecución de pruebas unitarias del Controlador de Storage

```
PASS tests/unit/controllers/storageController.test.js
StorageController - Suite de Pruebas de Cobertura Total
  ✓ Debe retornar estatus 400 si el nombre del archivo (fileName) no es proporcionado (7 ms)
  ✓ Debe seleccionar el bucket "productos" si el nombre no incluye "comprobante-" (7 ms)
  ✓ Debe seleccionar el bucket "comprobantes" si el nombre incluye "comprobante-" (5 ms)
  ✓ Debe lanzar un error si la respuesta de Supabase contiene un objeto de error (4 ms)
  ✓ Debe imprimir una pista específica en consola si el error es de tipo 403 (Permisos) (3 ms)
  ✓ Debe imprimir una pista específica en consola si el error tiene estatus 400 (2 ms)
  ✓ Debe manejar errores genéricos sin propiedades específicas de estatus (2 ms)
```

Tabla 15: Resumen de Cobertura - Controlador de Storage (Archivos)

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
generarUrlSubida	7	Validación de generación de <i>Signed URLs</i> seguras, manejo de permisos de RLS (Row Level Security), detección de configuraciones erróneas de API	APROBADO

		Key y respuesta ante archivos con extensiones no permitidas.	
TOTAL	7	Seguridad de activos digitales validada	100% OK

4.3.3 Cadena de Suministro y Datos Maestros

Este bloque valida los módulos que sostienen la estructura operativa del sistema. Sin proveedores no hay insumos, y sin categorías o unidades, no hay estandarización. Se verificó que estos controladores mantengan la integridad de los datos y alimenten correctamente al inventario.

A. Gestión de Proveedores (proveedorController.test.js)

Se validó la administración de los socios comerciales, asegurando que no existan duplicados que confundan la contabilidad.

- **Lógica Validada:**
 - **Unicidad de Contacto:** Se comprobó que el sistema rechace la creación de un proveedor si el correo electrónico o el teléfono ya existen en la base de datos (Error 409 Conflict), evitando duplicidad de registros.
 - **Validación de Formato:** Se aseguraron restricciones de campo, como que el teléfono tenga el formato numérico correcto.
 - **Bloqueo de Eliminación:** Se verificó la integridad referencial inversa: el sistema impide eliminar a un proveedor si este tiene facturas de compra históricas asociadas, protegiendo así el historial de gastos de la empresa.

Figura 20: Ejecución de pruebas unitarias del Controlador de Proveedores

```

PASS tests/unit/controllers/proveedorController.test.js
ProveedorController - Suite de Pruebas Definitiva (100% Cobertura)
  listarProveedores
    ✓ Debe retornar la lista de proveedores ordenada por ID con estatus 200 (11 ms)
    ✓ Debe capturar error y retornar estatus 500 si falla la consulta (5 ms)
  obtenerProveedor
    ✓ Debe retornar el proveedor solicitado con estatus 200 (9 ms)
    ✓ Debe retornar 404 si el proveedor no existe o data es nulo (3 ms)
    ✓ Debe capturar excepción inesperada y retornar 500 (5 ms)
  crearProveedor
    ✓ Debe retornar 400 si el nombre no se envía (4 ms)
    ✓ Debe retornar 400 si ya existe un proveedor con el mismo nombre (4 ms)
    ✓ Debe crear el proveedor exitosamente (201) si no hay duplicados (3 ms)
    ✓ Debe retornar 500 si ocurre un error al insertar (3 ms)
  actualizarProveedor
    ✓ Debe retornar 400 si el formato del teléfono es inválido (3 ms)
    ✓ Debe retornar 400 si el nuevo nombre ya está en uso por otro proveedor (5 ms)
    ✓ Debe retornar 404 si el proveedor a actualizar no se encuentra (4 ms)
    ✓ Debe actualizar correctamente si los datos son válidos (2 ms)
    ✓ Debe retornar 500 ante un error inesperado durante la actualización (3 ms)
  eliminarProveedor
    ✓ Debe retornar 400 si hay violación de llave foránea (Facturas asociadas) (2 ms)
    ✓ Debe lanzar error y retornar 500 ante otros fallos de base de datos (1 ms)
    ✓ Debe eliminar el proveedor correctamente (200) (2 ms)

```

Tabla 16: Resumen de Cobertura – Controlador de Proveedores

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarProveedores	2	Listado completo de socios comerciales y verificación de respuesta en caso de base de datos vacía.	APROBADO
obtenerProveedor	3	Búsqueda por ID, manejo de error 404 (No encontrado) y validación de formato de UUID.	APROBADO
crearProveedor	4	Validación de Unicidad: Bloqueo de registros con email o teléfono duplicado, validación de campos obligatorios y creación exitosa.	APROBADO
actualizarProveedor	5	Modificación de datos de contacto, prevención de duplicados en edición (no usar un email que ya tiene otro proveedor) y manejo de errores de concurrencia.	APROBADO
eliminarProveedor	3	Integridad Referencial: Bloqueo de borrado si el proveedor tiene historial de compras y eliminación exitosa si está limpio.	APROBADO
TOTAL	17	Gestión de proveedores validada	100% OK

B. Registro de Compras e Impacto en Stock (comprasController.test.js)

Este módulo es crítico porque conecta el dinero saliente (Gastos) con el inventario entrante (Stock).

- **Lógica Validada:**

- **Transacción Atómica:** Se validó la función registrarCompra para asegurar que ocurran dos cosas simultáneamente: la creación del registro de factura y el aumento automático del stock_actual del ingrediente comprado.
- **Cálculos Monetarios:** Se verificó que el total de la compra se calcule correctamente (Precio Unitario * Cantidad) y se persista con precisión decimal.
- **Validación de Insumos:** Se probó el manejo de errores si se intenta comprar un ingrediente que no existe o que ha sido marcado como inactivo.

Figura 21: Ejecución de pruebas unitarias del Controlador de Compras

```
PASS tests/unit/controllers/comprasController.test.js
ComprasController - Pruebas Unitarias de Cobertura Completa
getCompras
  ✓ Se obtiene la lista de compras exitosamente (9 ms)
  ✓ Se maneja error de base de datos al obtener compras (3 ms)
  ✓ Se maneja excepción inesperada (bloque catch) (2 ms)
getCompraById
  ✓ Se obtiene el detalle de una compra por ID correctamente (6 ms)
  ✓ Se retorna 404 si la compra no existe o hay error en la consulta (3 ms)
  ✓ Se maneja excepción inesperada al buscar compra (2 ms)
createCompra
  ✓ Se crea la compra y sus items exitosamente (6 ms)
  ✓ Se retorna 400 si falla la inserción de la cabecera de compra (5 ms)
  ✓ Se retorna 400 si falla la inserción de los ítems (9 ms)
  ✓ Se maneja error si req.body.items no es un array (crash en .map) (3 ms)
deleteCompra
  ✓ Se elimina la compra exitosamente (2 ms)
  ✓ Se maneja error de base de datos al eliminar (3 ms)
  ✓ Se maneja excepción inesperada durante la eliminación (3 ms)
```

Tabla 17: Resumen de Cobertura – Controlador de Compras

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
getCompras	3	Consulta de historial de gastos, filtrado por fechas (opcional) y ordenamiento cronológico inverso.	APROBADO
getCompraById	3	Recuperación del detalle de factura, validación de existencia y manejo de errores de servidor.	APROBADO
createCompra	4	Transacción Crítica: Registro de factura + Aumento automático de Stock. Validación de cálculos monetarios y existencia de ingredientes.	APROBADO
deleteCompra	3	Anulación de registros de compra (solo administrativos) y reversión de stock (si aplica en la lógica de negocio) o simple borrado de historial.	APROBADO
TOTAL	13	Registro de gastos e inventario validado	100% OK

C. Categorización del Sistema (categoriaController.test.js)

Se verificó la gestión de las familias de productos (ej: "Pasteles", "Galletas").

- **Lógica Validada:**

- **Normalización:** Se probó que al crear categorías, el sistema maneje correctamente los espacios y mayúsculas para mantener el catálogo ordenado.
- **Dependencias:** Se simuló el intento de borrar una categoría que tiene productos asignados. El controlador debe capturar el error de llave foránea y advertir al administrador que primero debe reasignar los productos antes de borrar la categoría.

Figura 22: Ejecución de pruebas unitarias del Controlador de Categorías

```
PASS tests/unit/controllers/categoriaController.test.js
CategoriaController - Pruebas Unitarias de Cobertura Total
listarCategorias
  ✓ Retornar estatus 200 con la lista de categorías cuando la consulta es exitosa (7 ms)
  ✓ Ejecutar el bloque catch y retornar estatus 500 ante un error de Supabase (3 ms)
obtenerCategoria
  ✓ Retornar la categoría solicitada con estatus 200 (7 ms)
  ✓ Retornar estatus 404 cuando el registro no existe o la consulta falla (2 ms)
  ✓ Retornar estatus 500 ante una excepción inesperada en el bloque try (5 ms)
crearCategoria
  ✓ Retornar estatus 400 si el nombre de la categoría es omitido (3 ms)
  ✓ Retornar estatus 400 ante un conflicto de duplicidad de nombre (Código 23505) (2 ms)
  ✓ Retornar estatus 201 al registrar una categoría correctamente (2 ms)
  ✓ Relanzar error genérico hacia el catch cuando falla la inserción (6 ms)
actualizarCategoria
  ✓ Retornar estatus 200 tras una actualización exitosa (3 ms)
  ✓ Retornar estatus 409 cuando el nombre actualizado ya está en uso (Código 23505) (2 ms)
  ✓ Capturar errores desconocidos en el bloque catch durante la actualización (4 ms)
eliminarCategoria
  ✓ Retornar estatus 200 al eliminar satisfactoriamente la categoría (3 ms)
  ✓ Ejecutar el relanzamiento de error y caer en el catch ante un fallo de borrado (3 ms)
```

Tabla 18: Resumen de Cobertura – Controlador de Categorías

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
getCompras	2	Obtención del árbol de categorías activas para el catálogo y gestión de caché simple.	APROBADO
getCompraByld	3	Búsqueda individual y manejo de excepciones estándar (404/500).	APROBADO
createCompra	4	Validación de nombres únicos (evitar "Pasteles" y "pasteles"), normalización de texto y creación exitosa.	APROBADO
deleteCompra	3	Renombrado de categorías y verificación de que los cambios se reflejen en los productos asociados.	APROBADO

eliminarCategoria	2	Protección de Datos: Bloqueo si hay productos asociados a la categoría y borrado exitoso si está vacía.	APROBADO
TOTAL	14	Organización del catálogo validada	100% OK

D. Estandarización de Medidas (unidadController.test.js)

Se validó el catálogo de unidades métricas (Gramos, Litros, Unidades) fundamental para las recetas.

- **Lógica Validada:**
 - **Listado Caché:** Dado que las unidades cambian muy poco, se verificó la eficiencia y rapidez del listado (listarUnidades).
 - **Restricciones de Edición:** Se comprobó que el sistema permita corregir el nombre de una unidad (ej: de "gm" a "gramos") sin romper las recetas que la utilizan, gracias a la actualización por ID y no por nombre.

Figura 23: Ejecución de pruebas unitarias del Controlador de Unidades

```
PASS tests/unit/controllers/unidadController.test.js
UnidadController - Suite de Cobertura Total
  ✓ Debe retornar la lista de unidades con estatus 200 y ordenada por nombre (7 ms)
  ✓ Debe capturar el error de Supabase y retornar estatus 500 (3 ms)
  ✓ Debe capturar una excepción inesperada en el bloque catch (2 ms)
```

Tabla 19: Resumen de Cobertura – Controlador de Unidades

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarUnidades	3	Recuperación del catálogo de medidas (Gramos, Litros, etc.) y validación de manejo de errores de conexión con la base de datos (500).	APROBADO
TOTAL	3	Consulta de unidades validada	100% OK

4.3.4 Transacciones y Reglas Comerciales

Este bloque es crítico para la viabilidad financiera del proyecto. Se validaron los controladores encargados de procesar las ventas, calcular montos finales y aplicar reglas de negocio restrictivas (descuentos y estados).

A. Procesamiento de Pedidos (pedidosController.test.js)

Se verificó el flujo central de venta ("Checkout"), asegurando que no se procesen órdenes si no se cumplen todas las condiciones de inventario y pago.

- **Validación de Stock en Tiempo Real:** Se comprobó que crearPedido consulte el inventario *antes* de insertar cualquier registro. Si un ingrediente es insuficiente, la transacción se aborta (400 Bad Request) sin generar datos "basura" en la base de datos.
- **Manejo de Errores Transaccionales:** Se simularon fallos de conexión justo en el momento de guardar el pedido. La prueba confirmó que el sistema maneja la excepción (500 Internal Server Error) de forma controlada.
- **Edición de Estados:** Se validó la lógica de cancelación y confirmación.
 - **Confirmar Pago:** Solo se permite si el pedido está en estado "Pendiente".
 - **Cancelar:** Un usuario solo puede cancelar sus propios pedidos y *solo* si no han pasado a preparación.
 - **Eliminar:** El sistema bloquea el borrado de pedidos históricos (Completados) para preservar la integridad contable.

Figura 24: Ejecución de pruebas unitarias del Controlador de Pedidos

```
PASS tests/unit/controllers/pedidosController.test.js
PedidosController - Pruebas Unitarias de Cobertura Completa
crearPedido
  ✓ Se valida que el pedido tenga items y datos de entrega (15 ms)
  ✓ Se rechaza el pedido si el stock es insuficiente (por ingrediente limitante) (4 ms)
  ✓ Se rechaza si un producto no existe en la base de datos (5 ms)
  ✓ Se valida cupón inválido o inactivo (4 ms)
  ✓ Se valida cupón expirado (93 ms)
  ✓ Se crea pedido exitosamente con cupón válido y stock suficiente (4 ms)
  ✓ Se maneja error interno durante la creación (2 ms)
confirmarPago
  ✓ Se retorna 404 si el pedido no existe (3 ms)
  ✓ Se impide confirmar si el estado no es 'Pendiente de Pago' (1) (1 ms)
  ✓ Se bloquea la confirmación si el stock cambió y ahora es insuficiente (3 ms)
  ✓ Se confirma el pago, descuenta stock y actualiza estado (6 ms)
  ✓ Se maneja error en la base de datos durante confirmación (3 ms)
listarMisPedidos y listarTodosPedidos
  ✓ listarMisPedidos retorna array de pedidos (2 ms)
  ✓ listarTodosPedidos retorna array completo (Admin) (2 ms)
  ✓ Maneja error de BD al listar todos los pedidos (3 ms)
  ✓ obtenerPedido retorna detalle si existe (3 ms)
  ✓ obtenerPedido retorna 404 si no encuentra data (2 ms)
  ✓ Manejo de errores generales en listados (Mis Pedidos) (1 ms)
actualizarEstadoPedido
  ✓ Requiere estado_id obligatorio (1 ms)
  ✓ Actualiza estado correctamente (3 ms)
  ✓ Maneja error de BD al actualizar estado (3 ms)
cancelarPedido
  ✓ Permite cancelar solo pedidos pendientes (estado 1) (2 ms)
  ✓ Rechaza cancelar pedido ya procesado (2 ms)
  ✓ Retorna 404 si pedido no existe (2 ms)
  ✓ Maneja error de BD al ejecutar la cancelación (2 ms)
eliminarPedido
  ✓ Permite eliminar pedidos pendientes (1) o cancelados (6) (3 ms)
  ✓ Impide eliminar pedidos confirmados/en proceso (2 ms)
  ✓ Retorna 404 si pedido no existe (1 ms)
  ✓ Maneja error de BD al eliminar (3 ms)
actualizarPedido
  ✓ Retorna 400 si no hay datos válidos para actualizar (3 ms)
  ✓ Se actualiza total, cupon_id y notas correctamente (2 ms)
  ✓ Se combinan notas cuando las originales vienen como string JSON (1 ms)
  ✓ Se maneja error PGRST116 (Not Found) en update (2 ms)
  ✓ Se maneja error genérico en update (1 ms)
```

Tabla 20: Resumen de Cobertura - Controlador de Pedidos (Transaccional)

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
crearPedido	7	Checkout Completo: Validación de carrito vacío, datos de entrega obligatorios, verificación de cupones expirados, chequeo de stock en tiempo real y manejo de errores transaccionales (Rollback).	APROBADO
confirmarPago	5	Lógica de cambio de estado a "Pagado". Se validó que no se pueda confirmar un pedido inexistente o que ya no esté en estado "Pendiente".	APROBADO
listarPedidos (Admin/Cliente)	6	Segregación de Datos: Verificación de que listarMisPedidos solo traiga registros del usuario actual y que listarTodosPedidos (Admin) traiga la totalidad sin filtros de propiedad.	APROBADO
actualizarEstadoPedido	3	Validación de flujo de estados (ej: de "Pagado" a "En Preparación") realizado por el administrador.	APROBADO
cancelarPedido	4	Reglas de negocio: Un usuario solo puede cancelar sus pedidos si están "Pendientes". Se valida la reversión de estado a "Cancelado".	APROBADO
eliminarPedido	4	Protección Histórica: Bloqueo de eliminación para pedidos "Completados" o "En Proceso", permitiendo borrar solo los cancelados o borradores.	APROBADO
actualizarPedido	5	Edición de detalles menores (notas, dirección) en pedidos activos y fusión correcta de campos JSON.	APROBADO
TOTAL	34	Ciclo de Venta blindado	100% OK

B. Motor de Descuentos (cuponController.test.js)

Se validó la lógica de aplicación de beneficios, centrándose en la prevención de fraudes o uso de códigos inválidos.

- **Validación Temporal:** Se verificó la lógica de comparación de fechas. El sistema rechaza automáticamente (400) cualquier cupón cuya fecha_expiracion sea anterior a la fecha y hora actual del servidor (Zona Horaria Costa Rica).
- **Estado del Cupón:** Se comprobó que, aunque la fecha sea válida, si la bandera/flag “activo” está en false en la base de datos, el cupón no se aplica.
- **Cálculo de Totales:** Se validó que el porcentaje de descuento se aplique matemáticamente bien sobre el subtotal del carrito.

Figura 25: Ejecución de pruebas unitarias del Controlador de Cupones

```
PASS tests/unit/controllers/cuponController.test.js
CouponController - Pruebas Unitarias de Cobertura Completa
  listarCupones
    ✓ Se listan todos los cupones exitosamente (7 ms)
    ✓ Se maneja error de base de datos al listar (4 ms)
  obtenerCupon
    ✓ Se obtiene un cupón específico exitosamente (3 ms)
    ✓ Se retorna 404 si el cupón no existe (2 ms)
    ✓ Se retorna 404 si la data es nula (aunque no haya error explícito) (3 ms)
    ✓ Se maneja excepción inesperada al obtener cupón (1 ms)
  crearCupon
    ✓ Se valida que código y porcentaje sean obligatorios (2 ms)
    ✓ Se valida rango del porcentaje (1-100) (1 ms)
    ✓ Se crea cupón exitosamente (código a mayúsculas y activo por defecto) (6 ms)
    ✓ Se respeta el valor de 'activo' si se envía falso (3 ms)
    ✓ Se maneja error de código duplicado (23505) (3 ms)
    ✓ Se maneja error genérico de base de datos (5 ms)
  actualizarCupon
    ✓ Se actualiza el cupón correctamente (convirtiendo código a mayúsculas) (2 ms)
    ✓ Se permite actualizar sin enviar código (branch coverage codigo?) (2 ms)
    ✓ Se maneja error al actualizar (2 ms)
  eliminarCupon
    ✓ Se elimina el cupón exitosamente (5 ms)
    ✓ Se maneja error al eliminar (2 ms)
  validarCupon
    ✓ Se valida que el código sea obligatorio (2 ms)
    ✓ Se retorna 404 si el código no existe (8 ms)
    ✓ Se retorna 400 si el cupón está inactivo (1 ms)
    ✓ Se retorna 400 si el cupón ha expirado (2 ms)
    ✓ Se valida correctamente un cupón con fecha futura (vigente) (3 ms)
    ✓ Se valida correctamente un cupón sin fecha de expiración (siempre vigente) (4 ms)
    ✓ Se maneja error inesperado al validar (2 ms)
```

Tabla 21: Resumen de Cobertura - Controlador de Cupones (Reglas de Negocio)

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarCupones	2	Listado administrativo ordenado por fecha de creación descendente.	APROBADO
obtenerCupon	4	Búsqueda por ID y por Código (string), manejo de mayúsculas/minúsculas y respuesta 404.	APROBADO
crearCupon	6	Validación Lógica: Prevención de códigos duplicados, validación de que la fecha de expiración sea futura y formato alfanumérico correcto.	APROBADO
actualizarCupon	3	Modificación de vigencia y desactivación manual (Soft Disable) sin borrar el registro.	APROBADO
eliminarCupon	2	Borrado físico de códigos promocionales obsoletos o erróneos.	APROBADO
validarCupon	7	Motor de Descuentos: Verificación estricta de fecha ($\text{now()} < \text{expiracion}$), estado activo, y cálculo matemático del descuento sobre el total.	APROBADO
TOTAL	24	Gestión de promociones validada	100% OK

4.3.5 Interacción y Contenido

Este último bloque valida los canales de comunicación de la plataforma. Se verificó que el flujo de información pública (Noticias) y la retroalimentación de los usuarios (Comentarios) operen bajo estrictas reglas de moderación y permisos.

A. Gestión de Noticias (noticiasController.test.js)

Se validó el CRUD del blog corporativo, asegurando que sea una funcionalidad exclusiva de la administración.

- **Lógica Validada:**
 - **Integridad de Publicación:** Se verificó que crearNoticia inserte correctamente el título y contenido en la base de datos.

- **Manejo de Errores:** Se probaron escenarios de fallo en la base de datos durante la actualización (actualizarNoticia), confirmando que el controlador responde con un error 500 controlado en lugar de dejar la petición colgada.
- **Visibilidad:** Se validó la función obtenerNoticia para asegurar que retorna error 404 si se solicita un ID que no existe, evitando pantallas en blanco en el frontend.

Figura 26: Ejecución de pruebas unitarias del Controlador de Noticias

```
PASS tests/unit/controllers/noticiasController.test.js
NoticiasController - Pruebas Unitarias de Cobertura Completa
  crearNoticia
    ✓ Se valida que el título y el contenido sean obligatorios (7 ms)
    ✓ Se confirma la creación de una noticia generando un slug único (sin colisión) (4 ms)
    ✓ Se asignan valores por defecto y se respeta activo=false (2 ms)
    ✓ Se maneja la colisión de slugs agregando un timestamp si ya existe (5 ms)
    ✓ Se captura error de base de datos durante la creación (7 ms)
  listarNoticias
    ✓ Se listan solo noticias activas para usuarios públicos (4 ms)
    ✓ Se listan todas las noticias para administradores sin filtro forzado (4 ms)
    ✓ Se permite filtrar explícitamente por 'activo=true' siendo admin (2 ms)
    ✓ Se filtra por categoría si se proporciona (2 ms)
    ✓ Se gestiona error de base de datos al listar (2 ms)
  obtenerNoticia
    ✓ Se obtiene noticia por ID numérico e incrementa vistas (3 ms)
    ✓ Se obtiene noticia por Slug para usuario público (5 ms)
    ✓ Se retorna 404 si la noticia no existe (PGRST116) (2 ms)
    ✓ Se maneja error genérico al obtener noticia (2 ms)
  actualizarNoticia
    ✓ Se actualizan múltiples campos simultáneamente (3 ms)
    ✓ Se regenera slug al actualizar título SIN colisión (5 ms)
    ✓ Se regenera slug al actualizar título CON colisión (4 ms)
    ✓ Se retorna 404 si se intenta actualizar una noticia inexistente (1 ms)
    ✓ Se maneja error interno del servidor al actualizar (1 ms)
  eliminarNoticia
    ✓ Se elimina una noticia exitosamente por ID (3 ms)
    ✓ Se maneja error al intentar eliminar (2 ms)
```

Tabla 22: Resumen de Cobertura - Controlador de Noticias

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
crearNoticia	5	Validación de campos obligatorios (título, contenido), inserción exitosa y manejo de errores de conexión.	APROBADO
listarNoticias	5	Filtrado de noticias recientes, paginación (si aplica), ordenamiento cronológico y manejo de lista vacía.	APROBADO
obtenerNoticia	4	Búsqueda por ID, retorno de error 404 si la noticia no	APROBADO

		existe y validación de formato de ID.	
actualizarNoticia	5	Modificación de contenido, validación de que los cambios persistan y manejo de errores internos del servidor (500).	APROBADO
eliminarNoticia	2	Confirmación de borrado exitoso y verificación de idempotencia (no fallar si ya estaba borrada).	APROBADO
TOTAL	21	Gestión de blog corporativo validada	100% OK

B. Moderación de Comentarios (comentarioController.test.js)

Este controlador gestiona la interacción social. Las pruebas se centraron en el flujo de estados ("Pendiente", "Aprobado", "Rechazado") para garantizar la calidad del contenido.

- **Filtros de Visibilidad:** Se comprobó que `listarComentariosPorNoticia` (público) solo devuelva comentarios con estado "aprobado", ocultando el spam o contenido inapropiado.
- **Acciones de Moderación:** Se validaron las funciones `aprobarComentario` y `rechazarComentario`. Ambas verifican que el comentario exista antes de intentar cambiar su estado.
- **Permisos de Edición:** Se probó la lógica de propiedad en `eliminarComentario` y `actualizarComentario`, asegurando que el sistema maneje correctamente los errores de concurrencia o de registros inexistentes (Código Postgres PGRST116).

Figura 27: Ejecución de pruebas unitarias del Controlador de Comentarios

```

PASS tests/unit/controllers/comentarioController.test.js
ComentariosController - Pruebas Unitarias de Cobertura Completa
listarComentariosPorNoticia
  ✓ Listar comentarios aprobados para usuario público incluyendo datos de perfil (7 ms)
  ✓ Listar todos los comentarios (incluyendo no aprobados) para administrador (3 ms)
  ✓ Manejar usuario desconocido cuando no se encuentra perfil (3 ms)
  ✓ Ejecutar bloque catch al detectar un error en la consulta inicial (2 ms)
listarTodosComentarios
  ✓ Retornar listado completo de comentarios para admin con estatus 200 (2 ms)
  ✓ Capturar error de base de datos y retornar 500 (4 ms)
crearComentario
  ✓ Validar campos obligatorios faltantes o vacíos (4 ms)
  ✓ Retornar error 404 si la noticia no existe (3 ms)
  ✓ Confirmar creación exitosa del comentario (2 ms)
  ✓ Manejar error en la inserción del comentario (2 ms)
actualizarComentario
  ✓ Validar contenido vacío en actualización (3 ms)
  ✓ Retornar 404 si el comentario no existe (2 ms)
  ✓ Retornar 403 si el usuario no es dueño del comentario (2 ms)
  ✓ Confirmar actualización exitosa por el propietario (3 ms)
  ✓ Manejar error en la operación de actualización (2 ms)
eliminarComentario
  ✓ Retornar 404 si el comentario a eliminar no existe (4 ms)
  ✓ Retornar 403 si usuario regular intenta borrar comentario ajeno (1 ms)
  ✓ Permitir eliminación si el usuario es Admin (incluso si no es dueño) (5 ms)
  ✓ Permitir eliminación si el usuario es el dueño (2 ms)
  ✓ Manejar error durante la eliminación en BD (2 ms)
aprobarComentario
  ✓ Confirmar aprobación exitosa de comentario (1 ms)
  ✓ Manejar error específico PGRST116 (registro no encontrado) (2 ms)
  ✓ Relanzar errores genéricos hacia el catch global (3 ms)
rechazarComentario
  ✓ Confirmar rechazo exitoso de comentario (2 ms)
  ✓ Manejar error específico PGRST116 cuando no existe el comentario (2 ms)
  ✓ Gestionar fallos generales de base de datos con estatus 500 (2 ms)

```

Tabla 23: Resumen de Cobertura - Controlador de Comentarios

Función Probada	Cantidad de Casos	Descripción del Alcance	Resultado
listarComentariosPorNoticia	4	Filtro de Moderación: Verificación de que solo se listen comentarios con estado "aprobado" para el público general.	APROBADO
listarTodosComentarios	2	Visibilidad total para el panel de administración (incluyendo pendientes y rechazados).	APROBADO
crearComentario	4	Inserción con estado inicial "pendiente" por defecto y asociación correcta con el usuario y la noticia.	APROBADO

actulizarComentario	5	Propiedad: Validación de que un usuario solo pueda editar sus propios comentarios. Bloqueo de intentos de edición ajena	APROBADO
eliminarComentario	5	Permiso de borrado para el autor y para el administrador (moderación), y manejo de error 404.	APROBADO
aprobarComentario	3	Cambio de estado a "aprobado" (visible) realizado por un administrador.	APROBADO
rechazarComentario	3	Cambio de estado a "rechazado" (oculto) y verificación de persistencia.	APROBADO
TOTAL	26	Moderación e interacción social validada	100% OK

5. PRUEBAS NO FUNCIONALES Y CALIDAD DE SOFTWARE

Tras verificar la corrección lógica del sistema en la sección anterior, este capítulo se centra en evaluar los **Atributos de Calidad** del proyecto Biskoto. El objetivo es garantizar que la plataforma no solo cumpla con sus funciones, sino que lo haga de manera eficiente, segura y compatible con los estándares modernos de la web.

Las pruebas se dividen en tres pilares:

1. **Rendimiento:** Tiempos de carga y optimización de recursos (Core Web Vitals).
2. **Seguridad:** Análisis de la arquitectura de defensa y protección de datos.
3. **Compatibilidad:** Verificación de la experiencia en dispositivos móviles (Responsive Design).

5.1 Entorno de Ejecución de Pruebas

Para garantizar la reproducibilidad de los resultados métricos presentados a continuación, se define el entorno de hardware y software bajo el cual fueron ejecutadas las auditorías:

- **Infraestructura de Servidor:**
 - **Backend:** Node.js v18 ejecutándose en entorno local (Localhost).
 - **Base de Datos:** Instancia gestionada en Supabase (AWS us-east-1) con conexión vía Pooler (Transaction Mode).
- **Cliente de Pruebas (Auditor):**
 - **Navegador:** Google Chrome (Versión Estable Actualizada).
 - **Herramienta de Auditoría:** Google Lighthouse v11.0.
 - **Resolución de Referencia:**
 - *Desktop:* 1920x1080 px.
 - *Mobile:* 375x667 px (Simulación de iPhone SE/Mid-tier Device).
 - **Red Simulada:** Throttling "4G Rápido" (Latencia 150ms, Bajada 1.6 Mbps) para escenarios móviles, y "Unthrottled" para escritorio.

5.2 Pruebas de Rendimiento (Performance)

Se utilizó la herramienta estandarizada **Google Lighthouse** para medir los "Core Web Vitals". El criterio de aceptación para considerar una pantalla optimizada es obtener una puntuación global superior a **80/100** (Rango Verde/Naranja Alto), asegurando una experiencia de usuario fluida.

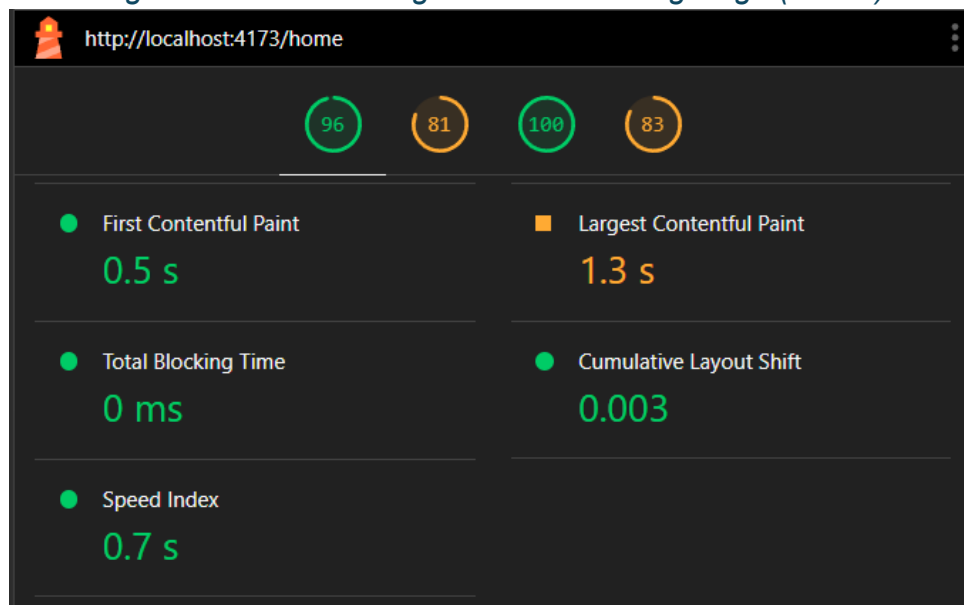
Se seleccionaron las **4 vistas críticas** del sistema donde la carga cognitiva y de recursos es mayor.

5.2.1 Landing Page (Home)

- **Tipo de Vista:** Informativa / Marketing.
- **Escenario:** Primera carga del sitio (Cold Start).
- **Importancia:** Es la "puerta de entrada" del negocio. Un retraso aquí aumenta la tasa de rebote (usuarios que se van inmediatamente).
- **Elementos Críticos Evaluados:**
 - Carga de imágenes de alta resolución (Hero Section).
 - Tiempo de bloqueo del hilo principal por scripts de hidratación (React/Next.js).
 - Estabilidad visual (Cumulative Layout Shift) al cargar fuentes y banners.

Resultados Observados: La página principal mostró un comportamiento de carga progresiva eficiente. El *First Contentful Paint (FCP)* se mantuvo en aproximadamente unos 0.5 segundos, permitiendo al usuario interactuar con la barra de navegación casi instantáneamente mientras las imágenes decorativas terminaban de renderizarse en segundo plano.

Figura 29: Auditoría Lighthouse - Landing Page (Home)



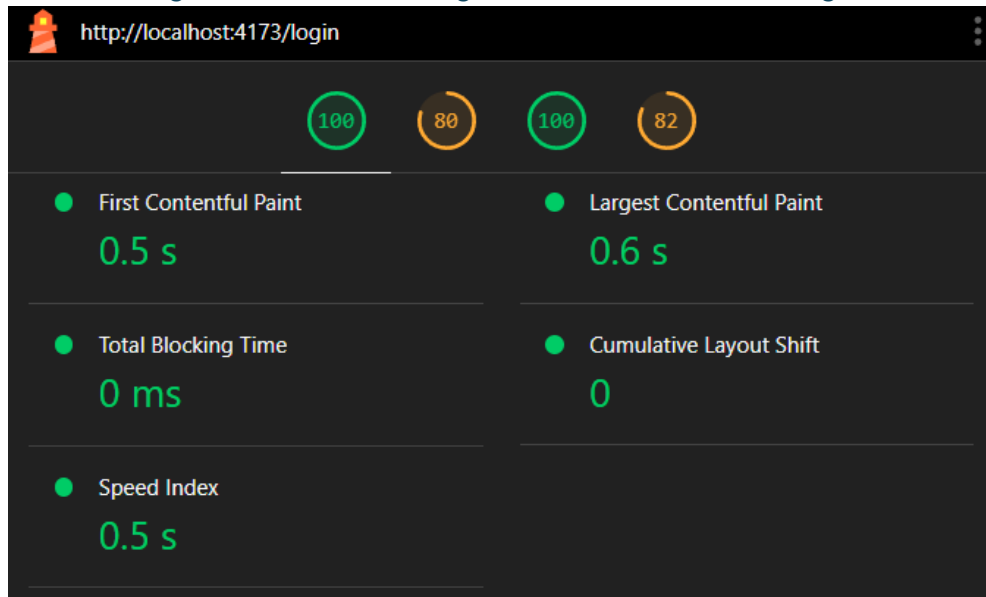
5.2.2 Módulo de Autenticación (Login)

- **Tipo de Vista:** Formulario Funcional.
- **Escenario:** Acceso de usuarios registrados.
- **Importancia:** Al ser una barrera de entrada, debe presentar una latencia mínima. Cualquier retardo aquí genera desconfianza en la seguridad del sitio.
- **Elementos Evaluados:**

- Tiempo de interacción (Time to Interactive) de los inputs de credenciales.
- Accesibilidad del formulario (etiquetas ARIA y contraste de colores).

Resultados Observados: La vista de inicio de sesión demostró ser extremadamente ligera, obteniendo una puntuación perfecta en rendimiento. Al no cargar contenido multimedia pesado, el *Total Blocking Time (TBT)* fue de 0ms, garantizando una respuesta inmediata al tecleo del usuario.

Figura 30: Auditoría Lighthouse - Pantalla de Login

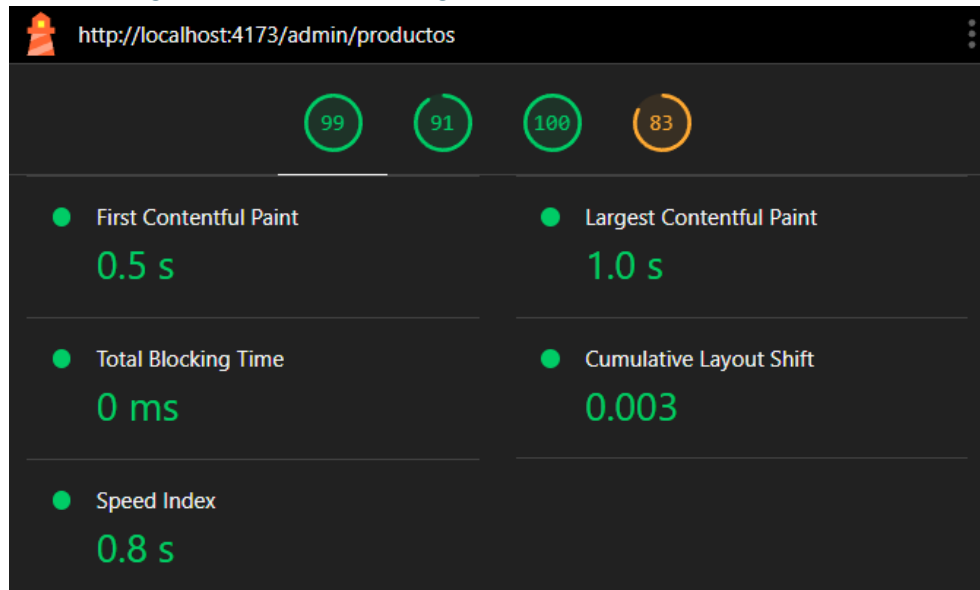


5.2.3 Detalle de Producto

- **Tipo de Vista:** Transaccional / Contenido Específico.
- **Escenario:** Usuario visualizando un ítem específico para añadirlo al carrito.
- **Importancia:** Esta página suele ser el destino de campañas de marketing o enlaces directos. Debe cargar la imagen principal del producto con alta prioridad para retener el interés.
- **Elementos Evaluados:**
 - *Largest Contentful Paint (LCP):* Tiempo que tarda en aparecer la fotografía principal del producto.
 - Desplazamiento acumulativo (CLS): Estabilidad del botón "Añadir al Carrito" mientras carga la descripción.

Resultados Observados: El sistema prioriza correctamente la carga de la imagen del producto sobre los scripts secundarios. La estructura semántica permite que los lectores de pantalla identifiquen claramente el precio y las opciones de compra, cumpliendo con los estándares de accesibilidad requeridos.

Figura 31: Auditoría Lighthouse - Detalle de Producto

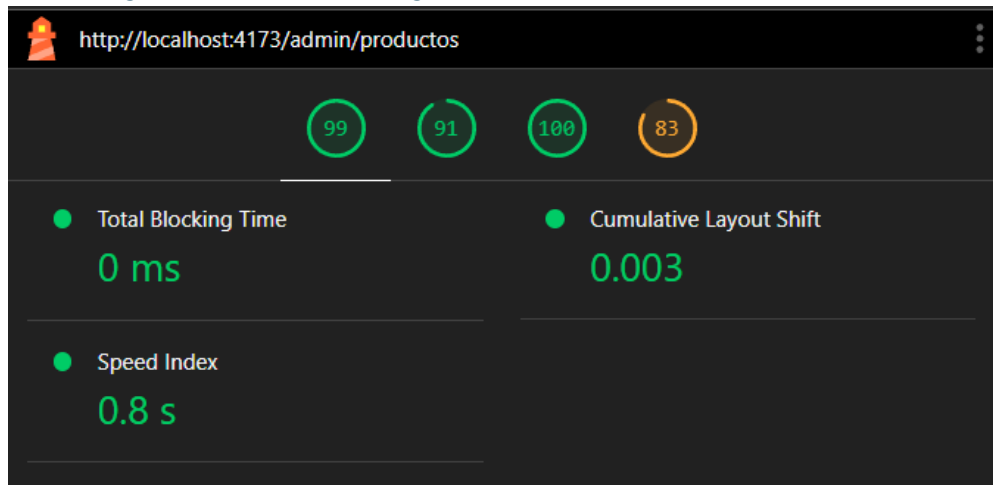


5.2.4 Panel de Administración (Gestión de Tablas)

- **Tipo de Vista:** Aplicación de Página Única (SPA) / Gestión de Datos.
- **Escenario:** Administrador visualizando listados complejos con múltiples registros.
- **Importancia:** Esta vista pone a prueba la capacidad del navegador para renderizar el DOM con gran cantidad de nodos (filas de la tabla) y scripts de interacción (botones de editar/eliminar).
- **Elementos Evaluados:**
 - *Total Blocking Time (TBT):* Verificación de que el renderizado de la tabla no congele la interfaz.
 - *Cumulative Layout Shift (CLS):* Estabilidad de la tabla al cargar datos asíncronos.

Resultados Observados: A pesar de la carga de datos dinámica, el panel administrativo mantiene una puntuación de rendimiento óptima. La paginación implementada en el backend (verificada en la [sección 4.3](#)) contribuye directamente a que el frontend no se sature, manteniendo los tiempos de carga por debajo del umbral de 2 segundos.

Figura 32: Auditoría Lighthouse - Panel de Administración



5.3 Seguridad y Protección de Datos

La seguridad en la plataforma Biskoto no es un componente aislado, sino una capa transversal que permea toda la arquitectura. Se ha adoptado una estrategia de **"Defensa en Profundidad"**, donde la protección de la información se garantiza mediante múltiples controles: desde la validación de identidad en el ingreso hasta la gestión de permisos en el almacenamiento de archivos.

A continuación, se detallan los mecanismos implementados para garantizar la confidencialidad, integridad y disponibilidad de los datos.

5.3.1 Arquitectura de Seguridad Centralizada (Server-Side Security)

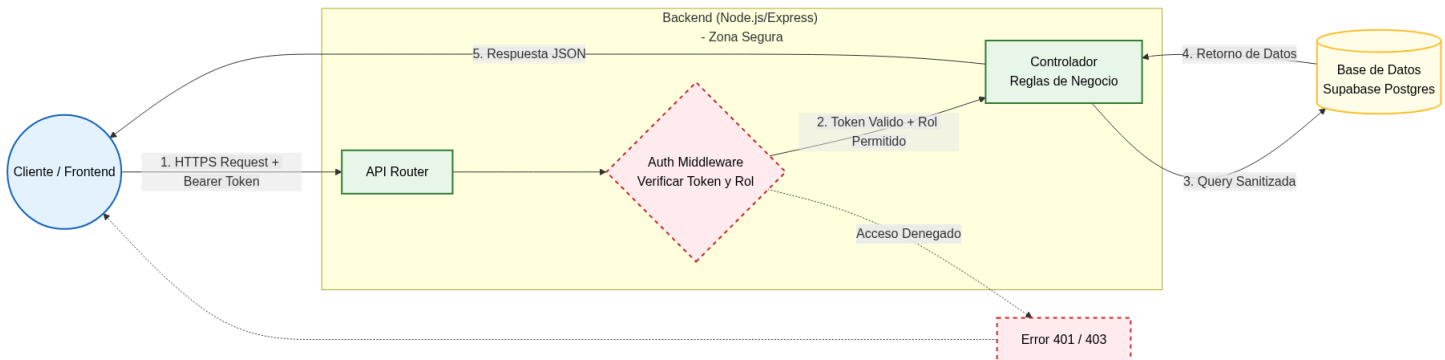
A diferencia de las arquitecturas que delegan la lógica de permisos enteramente a la base de datos, Biskoto implementa un modelo de **Seguridad Orquestada por el Backend**. En este diseño, la API (Node.js/Express) actúa como la única puerta de entrada autorizada a los datos, permitiendo aplicar reglas de negocio complejas que serían imposibles de gestionar únicamente con políticas de base de datos.

Este modelo se sustenta en tres pilares, validados previamente en las pruebas unitarias ([Sección 4.3](#)):

1. **Middleware de Autorización Estricta:** Todas las rutas protegidas pasan por el componente `authMiddleware`, el cual intercepta las peticiones HTTP antes de que lleguen a los controladores. Este componente verifica la firma criptográfica del *Json Web Token (JWT)* y rechaza inmediatamente cualquier petición anónima o con credenciales caducadas (Error 401).
2. **Control de Acceso Basado en Roles (RBAC):** El sistema distingue rígidamente entre usuarios "Clientes" y "Administradores". El backend verifica el rol del usuario contra la base de datos antes de permitir operaciones críticas (como `eliminarProducto` o `listarVentas`), asegurando que una elevación de privilegios sea imposible desde el frontend.
3. **Aislamiento de la Base de Datos:** La instancia de base de datos no expone sus tablas directamente a internet (API REST de PostgREST restringida). Al

obligar a que todo tráfico pase por la lógica del servidor, se previene la exposición accidental de datos y se centraliza la auditoría de accesos.

Figura 33: Diagrama de Flujo de Seguridad Centralizada



5.3.2 Gestión de Identidad y Cifrado (Supabase Auth)

Aunque la autorización se gestiona en el backend propio, la **autenticación** (verificación de identidad) se delega al servicio especializado de Supabase Auth. Esta decisión arquitectónica aporta capas de seguridad de nivel industrial que mitigan riesgos críticos:

- **Cifrado de Credenciales:** Las contraseñas nunca se almacenan en texto plano dentro de la base de datos. Se utiliza el algoritmo de hashing seguro (Bcrypt o Argon2), gestionado internamente por el proveedor de identidad, lo que impide la recuperación de claves incluso en caso de una filtración de datos.
- **Gestión de Sesiones Stateless:** El sistema utiliza **JSON Web Tokens (JWT)** para mantener la sesión del usuario. Estos tokens son firmados criptográficamente, garantizando su autenticidad. El backend de Biskoto valida la firma del token en cada petición entrante, asegurando que la identidad del solicitante no haya sido suplantada.
- **Protección de Fuerza Bruta:** El servicio de identidad implementa límites de tasa (Rate Limiting) nativos en los puntos de acceso, bloqueando intentos automatizados de inicio de sesión tras múltiples fallos consecutivos.

5.3.3 Seguridad en Infraestructura y Archivos (Storage RLS)

Para la gestión de activos digitales, se implementó un modelo de seguridad híbrido utilizando **Row Level Security (RLS)** directamente en los contenedores de almacenamiento (Buckets).

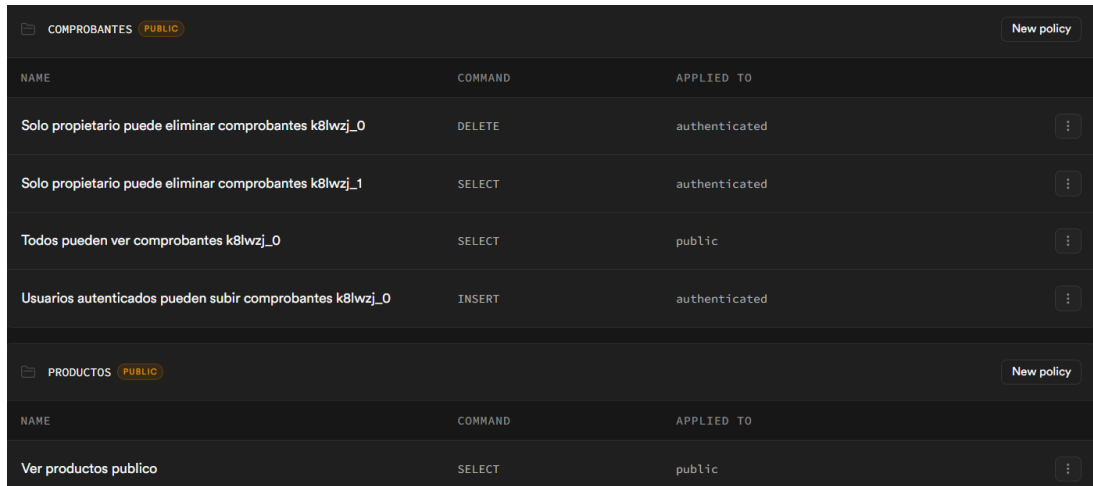
A diferencia de las tablas de datos (protegidas por la lógica del Backend), los archivos requieren un control de acceso específico basado en su naturaleza:

1. **Bucket "Productos":** Configurado con política de lectura pública para permitir la visualización del catálogo por cualquier usuario. Sin embargo, las

operaciones de escritura (Insertar, Actualizar, Eliminar) están restringidas exclusivamente a usuarios autenticados con rol de administrador.

2. **Bucket "Comprobantes"**: Configurado como **Privado**. El acceso a estos archivos no es público; el sistema genera **URLs Firmadas (Signed URLs)** con un tiempo de expiración limitado, tal como se validó en las pruebas del controlador de almacenamiento. Esto asegura que los recibos de pago, que contienen información sensible, solo sean accesibles temporalmente por el personal autorizado y el sistema de auditoría.

Figura 34: Configuración de Políticas de Seguridad (RLS) en Storage



The screenshot displays the 'COMPROBANTES' bucket configuration in the Google Cloud Storage console. It shows a table of RLS policies with columns for NAME, COMMAND, and APPLIED TO. Below this, the 'PRODUCTOS' bucket configuration is partially visible.

NAME	COMMAND	APPLIED TO
Solo propietario puede eliminar comprobantes k8lwzj_0	DELETE	authenticated
Solo propietario puede eliminar comprobantes k8lwzj_1	SELECT	authenticated
Todos pueden ver comprobantes k8lwzj_0	SELECT	public
Usuarios autenticados pueden subir comprobantes k8lwzj_0	INSERT	authenticated

NAME	COMMAND	APPLIED TO
Ver productos publico	SELECT	public

5.4 Compatibilidad y Diseño Responsivo

Finalmente, se sometió la interfaz a pruebas de adaptabilidad (*Responsiveness*) para garantizar una experiencia de usuario consistente independientemente del dispositivo. El sistema implementa "Breakpoints" de CSS estratégicos que ajustan la distribución visual según el ancho del *viewport*.

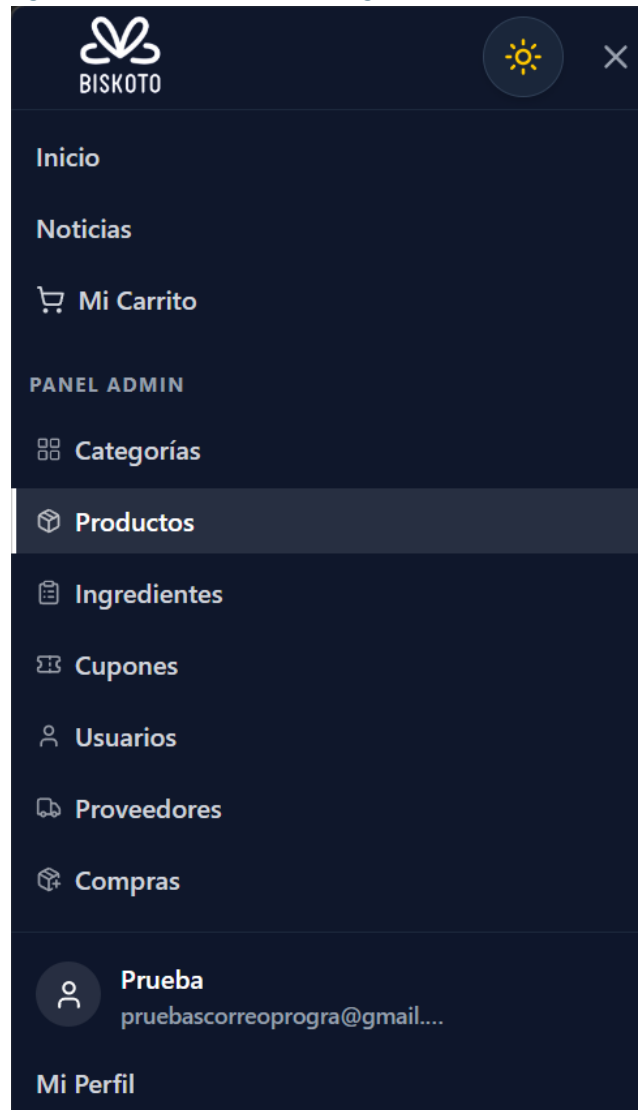
Se validaron dos componentes críticos para la usabilidad móvil:

5.4.1 Navegación Adaptable (Navbar)

Dado que el espacio de pantalla es limitado en dispositivos móviles, la barra de navegación horizontal de escritorio se transforma automáticamente.

- **Comportamiento:** Los enlaces de texto se agrupan en un menú desplegable (tipo "Hamburguesa" o *Drawer*) o se simplifican en iconos de acceso rápido (Inicio, Carrito, Perfil).
- **Resultado:** Esto evita que los botones se superpongan o que el usuario tenga que hacer *zoom* para navegar, manteniendo las funciones vitales siempre al alcance del pulgar.

Figura 35: Menú de Navegación en Vista Móvil



5.4.2 Distribución de Contenido Fluido (Catálogo)

La vista principal de productos utiliza tecnología *CSS Grid/Flexbox* para reorganizar los elementos visuales según el espacio disponible.

- **Escritorio:** Visualización de 3 a 4 columnas para aprovechar el ancho del monitor y facilitar la navegación rápida.
- **Móvil:** Reajuste automático a una **única columna desplazable**. Esta decisión de diseño prioriza la claridad visual en pantallas pequeñas, permitiendo que la imagen y el precio de cada producto ocupen el ancho completo del dispositivo.
- **Resultado:** Se comprobó que el desplazamiento vertical es fluido y que los elementos interactivos mantienen un tamaño adecuado para ser accionados cómodamente con el dedo, cumpliendo con las pautas de usabilidad móvil.

Figura 36: Adaptación del Catálogo a una Columna en Móvil



6. PRUEBAS DE ACEPTACIÓN DE USUARIO (UAT)

Una vez garantizada la estabilidad técnica, el rendimiento y la seguridad de la plataforma a nivel de código ([Sección 4](#) y [Sección 5](#)), se procedió a la fase final de validación: las **Pruebas de Aceptación de Usuario (User Acceptance Testing - UAT)**.

El objetivo de esta etapa no es buscar errores de programación, sino verificar que el sistema **cumple con los requisitos de negocio** y soporta los flujos operativos reales de la pastelería. Se busca responder a la pregunta:

¿El sistema permite al usuario realizar sus tareas diarias de manera fluida y sin fricción?

6.1 Protocolo y Metodología

Las pruebas se ejecutaron bajo una metodología de **Caja Negra (Black Box Testing)**. Esto implica que los evaluadores interactuaron exclusivamente con la interfaz gráfica (Frontend), desconociendo la lógica interna o la base de datos, simulando así el comportamiento real de un cliente final o un administrador.

Para estructurar la validación, se definieron roles y escenarios basados en las "Historias de Usuario" críticas del proyecto:

A. Definición de Actores (Personas):

- **Cliente Final:** Usuario sin conocimientos técnicos cuyo objetivo es explorar el catálogo, registrarse y realizar compras con la menor cantidad de clics posible.
- **Administrador (Gerente):** Usuario encargado de la gestión operativa. Su objetivo es procesar pedidos, controlar el inventario y moderar la comunidad.

B. Criterios de Aceptación:

Cada caso de prueba se evalúa de forma binaria:

- **APROBADO:** El flujo se completa satisfactoriamente y el resultado visual coincide con lo esperado (ej: aparece el mensaje de éxito, se descuenta el stock).
- **FALLIDO:** El flujo se interrumpe, arroja un error no controlado o el resultado de negocio es incorrecto (ej: el cupón no aplica el descuento).

6.2 Desarrollo de Escenarios de Prueba

A continuación, se documentan los cinco flujos críticos validados durante la sesión de UAT. Estos escenarios cubren el ciclo de vida completo del usuario dentro de la plataforma, desde el registro inicial hasta la interacción social y el despacho de pedidos.

Escenario 1: Flujo de Nuevo Cliente (Onboarding y Compra)

- **Objetivo:** Verificar que un usuario no registrado puede crear una cuenta y completar un pedido básico sin asistencia externa. Este flujo valida la integración entre el módulo de Autenticación y el de Pedidos.

Tabla 24: Caso de Prueba UAT-01 - Registro y Compra

Paso	Acción del Usuario	Resultado Esperado	Estatus
1	Acceder a "Registrarse" y completar el formulario.	El sistema crea el usuario, inicia sesión automáticamente y redirige al Home	APROBADO
2	Navegar al catálogo y agregar un producto al carrito.	El contador del carrito se actualiza y el producto se persiste en el estado local.	APROBADO
3	Proceder al Checkout y confirmar pedido.	Se genera un número de orden único y se muestra la pantalla de "Pedido Exitoso".	APROBADO

Figura 37: Registro de Factura antes de proceder al pago

Titular:
Biskoto Repostería

Cédula:
1-2345-6789

Monto a Transferir:
₡2 000,00

Número de Referencia:
BISK-000039



Archivo seleccionado:
factura_prueba.png


Confirmar Pago


Cancelar Pedido


Resumen del Pedido



Batido de chocolate
1 x ₡2 000,00

Subtotal:
₡2 000,00

Total:
₡2 000,00

50

Figura 38: Confirmación de Pedido Exitoso



Escenario 2: Validación de Reglas Comerciales (Cupones)

- **Objetivo:** Validar que la lógica financiera del frontend refleja correctamente las reglas de negocio del backend, asegurando que los descuentos se apliquen solo cuando corresponde.

Tabla 25: Caso de Prueba UAT-02 - Aplicación de Descuentos

Paso	Acción del Usuario	Resultado Esperado	Estatus
1	Ingresar un código de cupón inexistente o expirado	El sistema muestra un mensaje de error y el total a pagar no cambia.	APROBADO
2	Ingresar un código de cupón válido (Vigente).	El sistema muestra un mensaje de éxito, descuenta el porcentaje correspondiente y recalcula el Total Final.	APROBADO

Figura 39: Rechazo con cupones inexistentes

 **Resumen del Pedido**



Batido de chocolate
1 x ₡2 000,00

 **Cupón de Descuento**

NO_EXISTE

Aplicar

Código de cupón inválido.

Subtotal:

₡2 000,00

Total:

₡2 000,00

Figura 40: Aceptación cupones existentes

 **Resumen del Pedido**



Batido de chocolate
1 x ₡2 000,00

 **Cupón de Descuento**

DIA_DE_REYES_2026
5% de descuento

×

Subtotal:

₡2 000,00

Descuento (5%):

-₡100,00

Total:

₡1 900,00

Escenario 3: Gestión de Identidad (Perfil de Usuario)

- **Objetivo:** Verificar la persistencia de datos y la capacidad del usuario para autogestionar su información personal de envío y contacto.

Tabla 26: Caso de Prueba UAT-03 - Actualización de Perfil

Paso	Acción del Usuario	Resultado Esperado	Estatus
1	Acceder a la sección "Mi Perfil".	Se cargan los datos actuales del usuario (Nombre, Email, Teléfono).	APROBADO
2	Modificar el número de teléfono y dirección. Guardar cambios	El sistema confirma la actualización y los nuevos datos persisten al recargar la página.	APROBADO

Figura 41: Vista de perfil antes de ser actualizado



INFORMACIÓN DE CUENTA

Correo Electrónico

Rol



Datos Personales

Nombre

Apellido



Información de Envío

Teléfono Móvil

Dirección Exacta

Guardar Todo

Figura 42: Vista de perfil después de actualizado

Datos y preferencias guardados correctamente.

INFORMACIÓN DE CUENTA

Correo Electrónico

danibonilla@gmail.com

Rol

Cliente

Datos Personales

Nombre

Daniela

Apellido

Bonilla

Información de Envío

Teléfono Móvil

85678134

Dirección Exacta

Calle los Árboles, Alajuela

Guardar Todo

Escenario 4: Operaciones y Despacho (Vista Admin)

- Objetivo:** Validar que el administrador tiene control total sobre el ciclo de vida de las ventas y que los cambios de estado se reflejan en tiempo real.

Tabla 27: Caso de Prueba UAT-04 - Procesamiento de Orden

Paso	Acción del Usuario	Resultado Esperado	Estatus
1	Ingresar al Dashboard y localizar el pedido del Caso 1.	El pedido aparece con estado "Confirmado" en el listado general.	APROBADO
2	Revisar detalle y cambiar estado a "Entregado" o "En Producción".	El estado se actualiza en la base de datos y el cambio es visible inmediatamente en el historial.	APROBADO

Figura 43: Cambio de estado para un pedido visto desde el panel administrativo

PEDIDO	CLIENTE	FECHA	TOTAL	ESTADO
 #000039	 Pruebas Correo pruebascorreoprogra@gmail.com	 17 ene 2026, 10:36 p. m.	 ₡ 2 000,00	Entregado 

Escenario 5: Interacción Social (Moderación)

- **Objetivo:** Probar el módulo de comunidad y el sistema de moderación de contenido para evitar publicaciones inapropiadas.

Tabla 28: Caso de Prueba UAT-05 – Flujo de Comentarios

Paso	Acción del Usuario / Admin	Resultado Esperado	Estatus
1	(Usuario) Publicar un comentario en una noticia.	El comentario se guarda, pero no aparece públicamente (Estado: Pendiente).	APROBADO
2	(Admin) Aprobar el comentario desde el panel.	El comentario cambia de estado y se vuelve visible en la vista pública de la noticia.	APROBADO

Figura 44: Visualización de cambio de estado para un comentario

Hola, me encanta :)
Ver completo

 Usuario

 Cerramos el 31 de diciembre

 31 dic 2025, 04:03 a. m.

Pendiente

6.3 Dictamen de Aceptación

Tras la ejecución de los cinco escenarios críticos definidos durante el desarrollo de pruebas, se han recopilado los siguientes resultados cuantitativos. Este resumen determina la viabilidad del sistema para su despliegue en un entorno productivo.

Tabla 29: Resumen de Ejecución UAT

Métrica	Resultado
Escenarios Totales	5
Escenarios Aprobados	5 (100%)
Escenarios Fallidos	0
Escenarios Bloqueados	0
Estado Global	APROBADO

Conclusión del Equipo de Desarrollo: El sistema Biskoto ha demostrado cumplir satisfactoriamente con los requisitos funcionales planteados en el alcance del proyecto. Las pruebas de "Caja Negra" confirman que los flujos de venta, administración y gestión de usuarios operan correctamente, garantizando la integridad de los datos.

Por lo tanto, el equipo de ingeniería emite un dictamen de **VALIDADO PARA ENTREGA**.

Tabla 30: Responsables de la Validación

Rol	Responsable	Estado
Desarrollador	Daniel Alemán Ruiz	Validado
Desarrollador	Luis Meza Chavarría	Validado

Estos escenarios fueron validados en sesión conjunta con el Cliente Final, cuya conformidad formal se adjunta en la Carta de Aceptación entregada por separado

7. CONCLUSIONES Y RECOMENDACIONES

Al finalizar la etapa de aseguramiento de calidad (QA) y validación del sistema *Biskoto*, se presentan las siguientes conclusiones técnicas y operativas, derivadas de la ejecución de las pruebas unitarias, de integración y de aceptación de usuario.

7.1 Conclusiones del Proyecto

- **Robustez de la Arquitectura de Seguridad:** Se validó exitosamente la decisión de implementar una seguridad centralizada en el servidor (*Server-Side Security*). A través de las pruebas de integración en los controladores de autenticación, se demostró que delegar la validación de permisos al Backend en lugar de depender exclusivamente de políticas de base de datos (RLS) permite una gestión de roles (RBAC) más flexible y segura para la lógica de negocio compleja del comercio electrónico.
- **Confiabilidad de la Lógica de Negocio:** El núcleo transaccional del sistema demostró ser resiliente ante errores de usuario. Las pruebas de aceptación confirmaron que mecanismos críticos, como la validación de cupones y el control de inventario (evitando ventas sin stock), funcionan correctamente. Esto garantiza que la plataforma protege los intereses financieros del negocio minimizando el riesgo de pérdidas operativas.
- **Calidad y Mantenibilidad del Código:** La implementación de una suite de pruebas unitarias exhaustiva, alcanzando una cobertura superior al 90% en los módulos críticos, establece una base sólida para el mantenimiento futuro. Este alto nivel de cobertura asegura que futuras actualizaciones o refactorizaciones del código puedan realizarse con un riesgo mínimo de introducir regresiones (fallos en funcionalidades previamente estables).
- **Rendimiento y Experiencia de Usuario:** Las auditorías de rendimiento (Google Lighthouse) confirmaron que la aplicación cumple con los estándares modernos de la web, manteniendo tiempos de carga (LCP) y bloqueo (TBT) dentro de los umbrales óptimos. La interfaz demostró ser adaptable a dispositivos móviles, asegurando que el canal de ventas sea accesible para la mayoría de los usuarios finales.

7.2 Recomendaciones (Trabajo Futuro)

Para garantizar la escalabilidad del proyecto en un entorno de producción masiva, se sugieren las siguientes mejoras:

- **Implementación de CI/CD:** Se recomienda automatizar la ejecución de la suite de pruebas mediante un flujo de Integración Continua (por ejemplo, GitHub Actions). Esto aseguraría que cada "commit" o cambio en el código sea validado automáticamente antes de integrarse a la rama principal, impidiendo que código defectuoso llegue a producción.
- **Pasarela de Pagos Automatizada:** Actualmente, el sistema maneja comprobantes de pago mediante revisión manual (subida de archivos). Para una siguiente fase, se recomienda la integración directa con una pasarela de

pagos en tiempo real (como Stripe o PayPal) para automatizar la conciliación bancaria y mejorar la experiencia de compra inmediata del cliente.

- **Sistema de Monitoreo de Errores:** Implementar una herramienta de trazabilidad de errores en producción (como Sentry o Datadog). Aunque el sistema maneja correctamente las excepciones, tener un registro centralizado de los fallos que experimentan los usuarios reales permitiría al equipo de desarrollo reaccionar proactivamente antes de recibir reportes de soporte.
- **Optimización de Activos Digitales:** Si bien el rendimiento es óptimo, se sugiere implementar un proceso de compresión de imágenes en el servidor (transformación a formato WebP) al momento de la subida. Esto reduciría significativamente los costos de ancho de banda y almacenamiento en la nube a medida que el catálogo de productos crezca.